

Path-constrained trajectory planning for multi-robot manufacturing systems using null-space descent optimization with reduced Hessian

The International Journal of
Robotics Research
2026, Vol. 45(11) 1648–1681
© The Author(s) 2026
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/02783649251400381
journals.sagepub.com/home/ijr
 Sage

Xin Ye¹ , Max Grobbel¹, Tobias Schürmann¹ , Stefan Schwab¹ and Sören Hohmann²

Abstract

In a multi-robot manufacturing system, several robotic manipulators can be physically coupled and collaborate on a process along a given path. Due to the high degrees of freedom and the high-dimensional constraints for path tracking and physical coupling, it remains challenging to plan the motion trajectories for the robots toward the maximization of their collective capabilities, such as load capacity and stiffness. The state-of-the-art methods face conflicts between the optimality and computational complexity, as well as between constraint satisfaction and convergence speed. To resolve the conflicts, this paper proposes a single-stage optimization method set in order to solve for the optimal robot placement and the joint motion simultaneously under a given manufacturing path on the work-piece. This method set is centered upon a reduced Hessian optimization method that descends orthogonally to all the path and coupling constraints in a robot-specific null-space of reduced dimension, while also considering the motion limits. The convergence of the reduced Hessian method is proved mathematically. Furthermore, we verify the reduced Hessian method statistically against existing methods in numerical experiments of pulling a spring. To make the algorithm applicable to real-world robot-driven manufacturing processes with paths of arbitrary geometric complexity, the method set is extended with a parallel computing technique that deals with multiple connected path segments. Through real-world multi-segment milling experiments, we validate the concept of capability enhancement for the multi-robot manufacturing system under the usage of the proposed method set.

Keywords

path planning for manipulators, dual arm manipulation, cooperative manipulation, optimization-based motion planning, constrained motion planning, loop-closure constraints, trajectory-placement planning

Received: 16 January 2025; revised: 5 June 2025; accepted: 18 October 2025

Senior Editor: Ron Alterovitz

Associate Editor: Neil Dantam

Introduction

Robotic manipulators are utilized in various manufacturing processes thanks to their flexibility and versatility (Perzylo et al., 2019). In robot-driven 3D-printing, laser cutting, and welding, a high accuracy should be achieved on a pre-defined path; in metal bending, robots should possess high load capacity; in milling processes, high path accuracy and high stiffness under large process forces are required (Abele et al., 2006). In these manufacturing processes, paths are pre-given which define the relative movement of a tool to a work-piece. Robots should manipulate either the tool or the work-piece to realize the pre-defined relative movement. This relative movement may constrain all three translational and up to three rotational dimensions, but it specifies neither

the absolute movement of robots, nor their placement to the work-piece. Consequently, no start or end configurations are given, which prohibits the use of point-to-point trajectory generation methods. To make the robots execute the relative movement, both the placement and the joint trajectories of the robots should be planned within their reachable limits. Meanwhile, capabilities of robots such as accuracy, load

¹FZI Research Center for Information Technology, Karlsruhe, Germany

²Karlsruhe Institute of Technology (KIT), Karlsruhe, Germany

Corresponding author:

Xin Ye, FZI Research Center for Information Technology, Haid-und-Neu-Str. 10–14, Karlsruhe 76131, Germany.

Email: ye@fzi.de

capacity, manipulability, and stiffness should be enhanced according to process requirements.

When maximal achievable capabilities of a single robot are still insufficient for a given task, a multi-robot system with physical coupling can be created. Figure 1 shows such a setup for a milling process, which is our major use case. In the related research, it has been anticipated in Mühlbeier et al. (2021) that physically coupled robots, analogue to coupled springs, exhibit higher stiffness which is essential for milling processes. In the research field of cooperative manipulation, it has been shown that the load bearing capability of the multi-robot system is enhanced if multiple robots form a physical coupling by grasping a common object (Erhart and Hirche, 2016). We are willing to optimize these capabilities further toward the limits to leverage the full potentials of the physically coupled robots. Such an idea is illustrated in the two steps in Figure 2. Since the capabilities of robots are highly dependent on the joint configurations (Chiu, 1988), the redundant degree of freedom (DOF) in placement and joint configurations should be exploited for the optimization. As such, the problem setting of a path-constrained trajectory planning algorithm is the optimization of redundant DOFs toward the maximization of a configuration-dependent capability of the multi-robot system, while satisfying the physical coupling and path following constraints, as well as the motion limits.

The analysis of the state-of-the-art shows that the problem of multi-robot trajectory planning with path and coupling constraints is not solved with high performance. Most trajectory planning methods return sub-optimal results, because they do not exploit the redundant DOFs in the placement variables (Kalakrishnan et al., 2011; Zucker et al., 2013). Otherwise, the redundant DOFs are optimized separately, facilitating a complicated multi-level optimization problem (Nicola et al., 2018). To deal with the high-

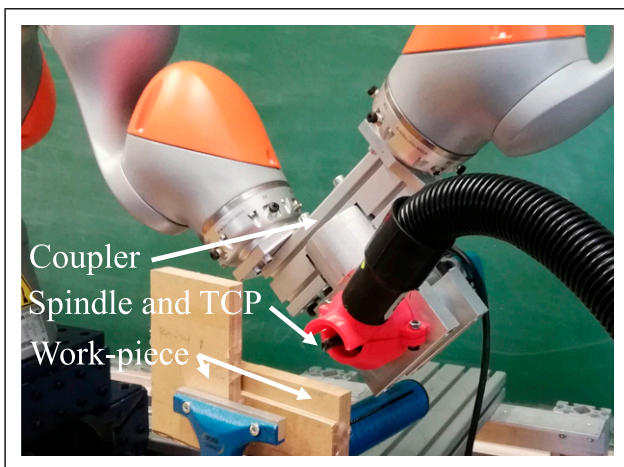


Figure 1. Two robots execute the milling process in a physically coupled configuration. The same image first appeared in Ye et al. (2025), and is reused under the authors' retained rights.

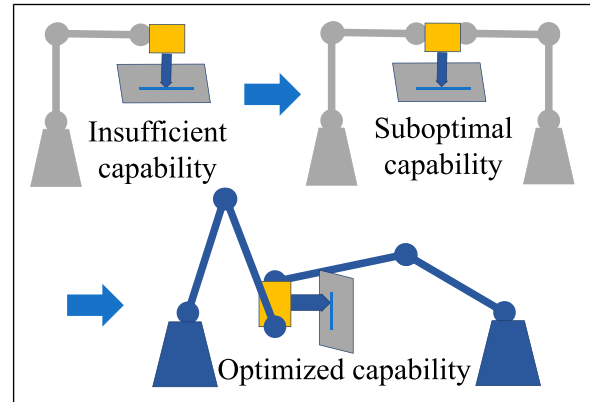


Figure 2. The idea with two steps to enhance capabilities of the robotic system. A physical coupling is achieved in the first step, while the redundant DOFs in placement and joint configurations are exploited in the second step.

dimensional path and coupling constraints, the available methods still suffer from slow convergence or labor-intensive multi-stage refinements (Kabir et al., 2021) (leading to 7.2 times longer computing time than our method in our test case), unless the optimality is partially sacrificed (Weingartshofer et al., 2023). These trade-offs have to be accepted in the existing methods.

We argue that a single-stage optimization algorithm can exhibit a high performance despite high dimensional decision variables and constraints. Such an algorithm conducts the gradient-based search in the feasible subspace that corresponds to the redundant DOF, where no local constraint violation occurs.

Accordingly, this paper builds upon the preliminary work (Ye et al., 2022) and makes the following contributions:

- An effective trajectory approximation method that requires neither feasible initial solutions nor trajectory refinements in post-processing
- A more rigorous reduced Hessian optimization method compared to Ye et al. (2022) for path-constrained placement and trajectory planning with a mathematically guaranteed convergence
- A parallel computing scheme to deal with multiple solution candidates and multi-segment manufacturing paths with high geometrical complexity
- The verification of the solution method with statistically high success rate, as well as the validation of the multi-segment stiffness-optimized trajectory planning for the real-world milling experiments.

The paper is organized as follows. After the introduction, the related work of path-constrained trajectory planning methods is investigated. Afterwards, we provide the preliminaries in the kinematic modeling of the multi-robot system, and then formulate the optimization problem

for the trajectory planning. The solution method set is illustrated afterwards with a convergence guarantee. Finally, we present extensive test results, before the conclusion is given.

Related work

In this section, the focus is on the solution approaches for trajectory planning and optimization for high DOF multi-robot systems. Especially, the suitability for manufacturing processes is analyzed.

Basic approaches for trajectory planning

The two most common approaches for trajectory planning are based on sampling and optimization, respectively.

Sampling-based approach. Methods belonging to the sampling-based approach start from a given configuration and search for collision-free via-points toward a goal configuration or goal set. Along these via-points, path and coupling constraint functions define a constraint manifold within the configuration space, as described in [Sintov et al. \(2018\)](#) and [Kingston et al. \(2019\)](#). The sampling is confined to the constraint manifold by either projection- or continuation-based methods. In projection-based sampling methods, a point is originally sampled in the configuration space without constraints by a general method such as Probabilistic Roadmap ([Kavraki et al., 1996](#)) or Rapidly-exploring Random Tree ([Jaillet and Porta, 2012](#); [LaValle and Kuffner Jr., 2001](#)). This point is then projected back to the constraint manifold, as in [Han \(2001\)](#) and [Kingston et al. \(2019\)](#). In continuation-based methods, the locally approximated tangent space of the constraint manifold is constructed, where the sampling takes place ([Yakey et al., 2001](#)).

In the basic problem setting of these sampling-based methods, collision avoidance and finding the shortest path, instead of the enhancement of robots' capabilities, are the focus, which are not aligned with the objective of the present paper. Moreover, the starting and ending points are determined separately (possibly optimized) prior to the sampling of via points in most sampling-based methods such as [Diaz Posada et al. \(2017\)](#); [Lu et al. \(2021\)](#). Determining different parts of decision variables in different stages makes the final result sub-optimal. Specifically, the optimization of, for example, the starting point is less meaningful for manufacturing processes as the robots might only start to but not fully engage the tool into the work-piece. At the following via points, however, the robots need a higher capability after the full engagement of the tool, but are already moving away from the optimal configuration occupied by the

starting point. A further disadvantage is the difficulty in dealing with the bounding constraints on the velocity and acceleration. A smoothening operation is needed after the sampling.

Optimization-based approach. Representative optimization-based methods with path and coupling constraints include Stochastic Trajectory Optimization for Motion Planning ([Kalakrishnan et al., 2011](#)), Covariant Hamiltonian Optimization for Motion Planning ([Zucker et al., 2013](#)), and Sequential Convex Optimization ([Bonalli et al., 2019](#); [Schulman et al., 2014](#)). However, these methods assume a pre-given starting configuration and a fixed placement, wasting the valuable DOFs for optimization. A few other optimization-based methods given as follows do leverage these DOFs by means of sampling, but still do not integrate them as decision variables of optimization. For 6-axis robots with a fixed placement, up to eight discrete starting configurations are acquired in [De Maeyer et al. \(2017\)](#). For robots with additional redundant joints in [Tang et al. \(2023\)](#), the angles of redundant joints are sampled in their movement range to find the configurations with the highest end-effector stiffness. In [Weingartshofer et al. \(2023\)](#), after sampling the possible placements and the first point on the path, a set of feasible initial configurations is obtained by solving the analytical inverse kinematics (AIK). Starting from this set, optimization problems toward the second way-points, from the second to the third, and so on are solved by parallel computing. These methods that sample the redundant DOFs have the common draw-back of a lack of possibility to further adjust the previously sampled DOFs in the phase of optimization. Consequently, a large number of samples are taken. For complicated manufacturing paths, those samples that later lead to non-reachable path points cannot be adjusted but only be discarded. As such, these methods suffer from a low scalability w.r.t. number of redundant DOFs and a low success rate.

Different from the above, the multi-stage optimization method in [Kabir et al. \(2021\)](#) optimizes the whole joint trajectories of all robots, including their starting points. This is done by parameterizing the trajectories with B-splines and taking all the control points as decision variables for an optimization problem. However, at least one initial solution of the joint trajectory must be obtained, whose feasibility is difficult to ensure in a highly constrained multi-robot system for complicated paths. Moreover, the multi-stage method requires more tuning efforts in multiple stages, and exhibits a low computational efficiency (14.6 s for one stage and 15 way-points in [Kabir et al. \(2021\)](#)).

Furthermore, the optimization-based approach has the common problem of running into local optima, which is

tackled by multiple random starts (Ugray et al., 2007) or more complicated meta-heuristic methods (Nicola et al., 2018).

Finally, we determine to select and improve the optimization-based approach despite all its challenges, because this basic approach is aligned with the objectives of the present paper in terms of optimizing robots' capabilities, leveraging all available DOFs, and integrating smooth trajectory parameterization techniques. The necessary aspects of improvement are further investigated in view of the state-of-the-art methods as follows.

Handling path and coupling constraints

In the optimization-based planning approach, simply giving constraint equations on discretized trajectory points to an optimization solver does not always work (leading to a success rate of only 33.26% in our test case). This is because of the nonlinearity of the robot kinematics in constraint equations, as well as the lack of control of trajectories between the discretized points. The following methods exist to mitigate those issues.

Manifold operations. The constraint equations of coupling and path tracking for multiple robots define manifolds in the joint space. When all constraints are satisfied, the joint motion stays within the corresponding constraint manifold. If this constraint manifold is endowed with a metric defining the distance between two points on it, one can leverage efficient operations of the Riemannian geometry, namely the exponential and logarithmic maps, as well as parallel transport (Pennec et al., 2006). Equipped with these operations, the manifold optimization methods such as Newton-Raphson, Conjugate Gradient, and Trust Region are highly efficient with available toolboxes (Boumal, 2023; Boumal et al., 2014). The manifold optimization has been utilized in parameter tuning within non-Euclidean spaces, such as learning impedance control parameters or optimizing the end-effector orientation for manipulability (Jaquier et al., 2020; Jaquier and Rozo, 2020). But path constraints inherently possess neither a Riemannian metric nor the efficient operations. In Zucker et al. (2013), a projection similar to the exponential map (without logarithmic map) is conducted in every iteration of trajectory planning to move the decision variables back to the constraint manifold after an unconstrained step. However, without being directed by the logarithmic map near the optimal trajectory, the unconstrained step might be nearly orthogonal to the constraint manifold, which leads to a smaller optimization progress and

causes chatters that has to be constantly canceled out. This impedes the convergence speed of the optimization.

Penalty functions. Beside the actual goal of enhancing capabilities in the manufacturing process such as stiffness, the objective function is already populated with terms to penalize abrupt joint angle changes, singularity, and collision with the environment. However, adding the path and coupling constraints into the objective function as penalty terms is still a common practice as in, for example, Kalakrishnan et al. (2011), Schulman et al. (2014), and Kabir et al. (2019). Cost weight scheduling, as proposed in Zucker et al. (2013), is thus indispensable. This has been developed into the penalty scaling in the sequential convex optimization for collision avoidance (Schulman et al., 2014), and further into the multi-stage successive constraint refinement for coverage-oriented placement planning in Malhan et al. (2019) and Kabir et al. (2021). In the latter case, the solution process is divided into two or more stages. In the first stage, only a part of the penalties that have fewer but unambiguous attraction regions, such as collision avoidance, are applied. Other penalty terms whose attraction regions have a scattered landscape such as position constraints are enabled in a later stage. For a case of traversing a small ring through a big one, the highest success rate of 63.28% is achieved, if the best penalty sequence among four penalty terms is chosen (Kabir et al., 2021). But for an inexperienced user, the multi-stage tuning is time-consuming with no guaranteed success rate, although a remedy with adaptive search on potentially more costly real experiments is proposed (Kabir et al., 2018).

Manifold learning. Most learning-based methods for trajectory planning with manifold constraints trade time-consuming offline training for fast online planning capabilities. Zha et al. (2018) learn the metric of the manifold represented by a graph, so as to select promising sampling direction for online planning. Combined with the projection operation, it results in a high success rate. Qureshi et al. (2021) implement an end-to-end learning from an environment encoding to the next sampling step. However, the above sampling-based manifold learning methods all face restrictions in the free determination of the placement and the starting configuration, which sacrifices the whole-path optimality. There exists an exception using an optimization-based method (Kicki et al., 2023). In this method, manifold constraints are relaxed as penalty terms, whose Lagrange multiplier is learned so as to facilitate fast planning and replanning in dynamic environments such as air hockey. This method does not handle constraint manifolds of path tracking.

Dealing with process-robot specifications

In manufacturing processes of complex work-piece geometry, the path given by Computer-Aided Manufacturing software can be arbitrarily long with large number of direction changes. After a discretization, with high amount of way-points on the Cartesian path to be tracked, the joint trajectories must also be parameterized with comparable amount of decision variables, owing to the limited approximation capabilities of the trajectory using, for example, B-splines (Lyche et al., 2018b; Takacs and Takacs, 2016). Due to the nonlinear scalability of most parameter optimization methods w.r.t. number of decision variables, it is no more economical to plan the corresponding trajectory as a whole. To allow a linear scalability, Weingartshofer et al. (2023) solve a series of simplified optimization sub-problems sequentially at the cost of the whole-path optimality. An alternative method proposed in Kabir et al. (2021) is to split the path into segments and to optimize for each, subjected to additional inter-segment continuity constraints. These constraints are then converted to a penalty term which serves as the guiding heuristic for a branch-and-bound search. These methods either suffer from computational complexity or sacrifice the whole-path optimality by simplifying the computation.

Due to the characteristics of robotic manipulators, the mapping from task space to joint space is not unique. The robots' capabilities, for example, stiffness, are often a measure in task space but dependent on joint configurations, which in turn depend on the robots' placement w.r.t. the work-piece. Several distant configurations with different placements might exhibit local optimality in desired capabilities. A genetic algorithm is applied in Bachmann et al. (2021) to determine the robot-workpiece placement considering reachability and collision avoidance. The intertwined placement and trajectories are considered in Menasri et al. (2013), Nicola et al. (2018), and Mutti et al. (2021) by means of nested meta-heuristic algorithms, leading to a high number of iterations and a prolonged computation time. The multi-start strategy by sampling random seeds of initial guesses and carrying out the optimization multiple times is utilized in Schulman et al. (2014), De Maeyer et al. (2017), Kabir et al. (2021), and Weingartshofer et al. (2023) to deal with the non-convex nature of the planning problem. However, for complex manufacturing paths, a random generation of initial solutions, even with the help of analytical inverse kinematics, results mostly in non-feasible ones, because not every point of the highly complex path can be reached by all robots.

Conclusion

Existing methods face trade-offs between the computational complexity and the whole-path optimality on the one hand, as well as between an accurate constraint satisfaction and a high convergence speed on the other hand. To the best of our knowledge, there is no method that utilizes both exponential and logarithmic map in the path-constrained trajectory optimization for robots, so as to facilitate a fast convergence with high accuracy of constraint satisfaction. An analogy to these mappings does exist as the horizontal and vertical step in Gabay (1982) and Kupfer (1996), which show the unexplored potential in our preliminary work (Ye et al., 2022). In addition, there exists no method dealing with multiple path segments that leverages parallel computing by means of the promising alternative direction method of multipliers (ADMM) (Boyd et al., 2011).

Preliminaries and system modeling

This section first deals with the preliminaries in the kinematic modeling for the trajectory planning of multi-robot manufacturing systems. Afterward, the stiffness of such systems, as one of their crucial capabilities in manufacturing processes, is modeled so that it can be later integrated in the optimization-based planning approach.

Basic geometry and kinematics

Suppose a multi-robot system of a number of robots n_{rob} has n_J joints in total. The trajectories of joint $j = 1, \dots, n_J$ can be represented as $q_j(u)$ with the parameter u , progressing in $u \in [0, 1]$. The trajectories of all joints are denoted as the vector $\mathbf{q}(u) = [q_1(u), \dots, q_{n_J}(u)]^T \in \mathbb{R}^{n_J}$. Apart from the joints, the placement of robot bases is represented by up to three translational and three rotational variables. The work-piece, either fixed in a stationary manner or on a robots' gripper, has its placement also represented by no more than six variables. So we define the static placement variables $\mathbf{y} \in \mathbb{R}^{n_{\text{pr}} + n_{\text{pw}}}$, where n_{pr} denotes the DOF for robot base placement and n_{pw} denotes that of the work-piece. We denote $\boldsymbol{\theta}(u) = [\mathbf{q}^T(u), \mathbf{y}^T]^T \in \mathbb{R}^{n_J + n_{\text{pr}} + n_{\text{pw}}} = \mathbb{R}^{n_\theta}$ as the configuration space trajectory, a combination of all the joint trajectories and the static placement variables.

The kinematics of path tracking and coupling are shown in Figure 3. A path $\mathbf{s}(\cdot) : [0, 1] \rightarrow \mathbb{R}^{n_{\text{path}}}$ with the dimension $n_{\text{path}} \leq 6$ defines the relative movement of the Tool Center Point (TCP) to a work-piece, dependent on $u \in [0, 1]$. The path error vector function $\mathbf{e}_{\text{path}}(u) = \mathbf{e}_{\text{path}}(\boldsymbol{\theta}(u), \mathbf{s}(u))$ of

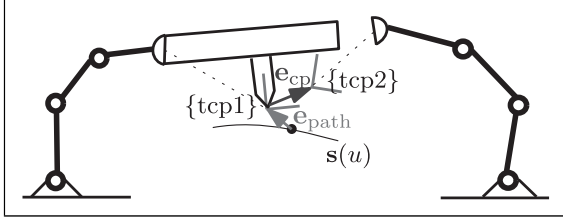


Figure 3. The kinematics with the path $s(u)$, the path tracking error $e_{\text{path}}(u)$, and the coupling error e_{cp} in the special case of $n_{\text{rob}} = 2$. The coupling error e_{cp} is defined between the body frames $\{\text{tcp1}\}$ and $\{\text{tcp2}\}$.

dimension n_{path} defines the deviation of the real TCP from the path. Hereby, the forward kinematics of the robots is necessary to transform $\theta(u)$ from the configuration space toward the TCP pose in the Cartesian space so that the pose can be compared with $s(u)$. Physical couplings with a coupling unit, that is, a coupler, also denoted as cp, constitute kinematic loop-closure constraints. A body frame of one robot should keep a prescribed relative pose of dimension n_{cp} to a body frame of the other robot. The coupling error vector function $e_{\text{cp}}(\theta(u)) \in \mathbb{R}^{n_{\text{cp}}}$ defines the deviation of the relative pose to the prescribed one. We denote the vector concatenation of e_{path} and e_{cp} as pose constraint errors:

$$e_{\text{pose}}(\theta(u), s(u)) = [e_{\text{cp}}(\theta(u))^T, e_{\text{path}}(\theta(u), s(u))^T]^T. \quad (1)$$

The constraints for the path tracking and the physical coupling are defined as a system of equations $e_{\text{pose}}(\theta(u), s(u)) = \mathbf{0}$.

In processes with a complex path, the path can be divided in n_{seg} segments as $s_m(\cdot)$, $m \in \{1, \dots, n_{\text{seg}}\}$, either manually or by algorithms such as those in Kabir et al. (2021) if required. In each segment m , the configuration space trajectory $\theta_m(u) = [q_m^T(u), y_m^T]^T$ is defined. The path-constrained trajectory planning problem is stated as follows for each individual segment:

Problem 1. (Path-constrained trajectory planning)

Given a path $s_m(\cdot) : [0, 1] \rightarrow \mathbb{R}^{n_{\text{path}}}$ from segment $m \in \{1, \dots, n_{\text{seg}}\}$, find a feasible configuration space trajectory $\theta_m(\cdot) = [q_m^T(\cdot), y_m^T]^T : [0, 1] \rightarrow \mathbb{R}^{n_j + n_{\text{pr}} + n_{\text{pw}}}$, satisfying the path following and loop-closure constraints:

$$e_{\text{pose}}(\theta_m(u), s_m(u)) = \mathbf{0}, \quad \forall u \in [0, 1] \quad (2)$$

while being kept within bounds:

$$y_{\min} \leq y_m \leq y_{\max} \quad (3)$$

$$q_{\min} \leq q_m(u) \leq q_{\max}, \quad \forall u \in [0, 1]. \quad (4)$$

By demand, (4) can be extended by its time derivatives up to any given order.

Across segments, the robots and the work-piece should not change their placements, and joint trajectories should exhibit continuity to ensure a smooth transition. Therefore, the multi-segment planning problem is defined as follows:

Problem 2. (Multi-segment planning)

Given an inter-connected multi-segment path $s_1(\cdot), \dots, s_{n_{\text{seg}}}(\cdot)$, find feasible configuration space trajectory segments $\theta_1(\cdot), \dots, \theta_{n_{\text{seg}}}(\cdot)$, satisfying the intra-segment constraints (2)–(4) $\forall m \in \{1, \dots, n_{\text{seg}}\}$, as well as the inter-segment constraints:

$$y_1 = y_2 = \dots = y_{n_{\text{seg}}} \quad (5)$$

$$q_m(1) = q_{m+1}(0), \forall m \in \{1, 2, \dots, n_{\text{seg}} - 1\}. \quad (6)$$

By demand of continuity, (6) may be extended by its time derivatives up to any given order.

Further, by considering metrics in the planning, one achieves an optimal solution that exploits the capabilities of multi-robot systems. In our major use case of the robot-driven milling, the stiffness against disturbances of process forces is the crucial capability.

Static stiffness

This subsection illustrates the modeling of the TCP static stiffness considering the elasticity of the robotic joints and of the coupler. We apply a torsional spring model (Abele et al., 2007; Busson et al., 2017) for the robot joints, and a spring network model (Curtin and Scher, 1990; Pan et al., 2019) on the coupler. Afterward, we analytically illustrate the influence of unknown coupler elasticity on the accuracy of the resulting TCP stiffness. This is due to the complexity of identifying the 3D elasticity parameters of the coupler. In the later formulation of the trajectory planning problem, we stay with the simple model of a rigid coupler and the elastic joints, as introduced in Ye et al. (2022). Nevertheless, the analysis provides an insight on the cause of differences between the predicted TCP stiffness and the reality from milling experiments.

Figure 4 shows the spring network model with an example of $n_{\text{rob}} = 2$. On the coupler, three points of interest are the two flanges connecting to the two robots and the TCP. The springs between these points possess 3D rotational and 3D translational elasticity. We derive

the analytical expression of the TCP stiffness depending on the elasticity of these abstracted springs.

A further simplification of the spring network is possible considering the mechanical construction of the real coupler, as shown in Figure 5(a). Because of the geometry of the profile, there are two weakest sections, as marked with the red dashed ovals. Only these two sections of the coupler are modeled with springs, as shown in the two-spring model in Figure 5(b). This makes the derivation of the TCP stiffness tractable under limited modeling error.

In each individual rigid-body tree of robot $r = 1, 2$, a TCP frame is defined as $\{tcp_r\}$. In the idle situation without any external loads (but with arbitrary internal preload and the corresponding deformation), $\{tcp_r\}$, $\forall r = 1, 2$ should overlap due to the physical coupling. In the situation with an external load, a wrench $w_{ext} \in \mathbb{R}^6$ is exerted on the real TCP. Due to the connection with robots, additional constraint wrenches w_1, w_2 occur in the static case. Under w_1 and w_2 , the robots with elastic joints undergo joint deformation, denoted with vectors δq_1 and δq_2 . This leads to the deviation of the frames $\{tcp1\}$ and $\{tcp2\}$ from the rigid-body model, denoted as $J_1 \delta q_1$ and $J_2 \delta q_2$, respectively. J_r is the Jacobian matrix of robot r w.r.t. $\{tcp_r\}$. The coupler also experiences a deformation δp_1 from the rigid-body frame $\{tcp1\}$ and a deformation δp_2 from the rigid-body frame $\{tcp2\}$. With all the above deformation, the real TCP frames from the view point of each robot still overlap, which is expressed by the following loop-closure condition:

$$J_1 \delta q_1 + \delta p_1 = J_2 \delta q_2 + \delta p_2. \quad (7)$$

The analysis up till now can be extended to any number of physically coupled robots $n_{rob} > 1$. While

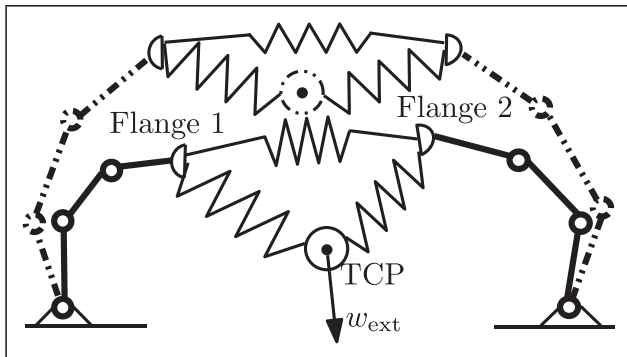


Figure 4. The coupler elasticity is modeled by a spring network, while the elasticity of robots is modeled with torsional springs in the joints. Without process wrench, the robots are at the configuration of dash-double-dotted lines. Under process wrench w_{ext} , the configuration is at the solid lines.

multiple robots are coupled in parallel, the connection of each individual robot to the potentially elastic section between the flange and TCP is serial. By applying the loop-closure condition and Hooke's law, and by eliminating the variables δq_r and δp_r , the following result of TCP-stiffness is obtained:

$$K_{tcp} = \sum_{r=1}^{n_{rob}} K_{rob,r} (I_6 + K_{c,r}^{-1} K_{rob,r})^{-1}, \quad (8)$$

whereby I_6 is the identity matrix of dimension 6. The matrix $K_{rob,r} = (J_r K_q^{-1} J_r^T)^{-1}$ stands for the stiffness of a single robot with K_q representing the constant joint stiffness and $J_r = J_r(\theta(u))$ the Jacobian depending on the configuration space trajectory. The matrix $K_{c,r}$ denotes the world-frame stiffness matrix of the coupler between TCP and the flange connected to the r -th robot. The world-frame stiffness matrix is calculated from the constant TCP-body-frame one ${}^b K_{c,r} \in \mathbb{R}^{6 \times 6}$ as follows:

$$K_{c,r} = \begin{bmatrix} R_{tcp}^w & \\ & R_{tcp}^w \end{bmatrix} K_{c,r}^b,$$

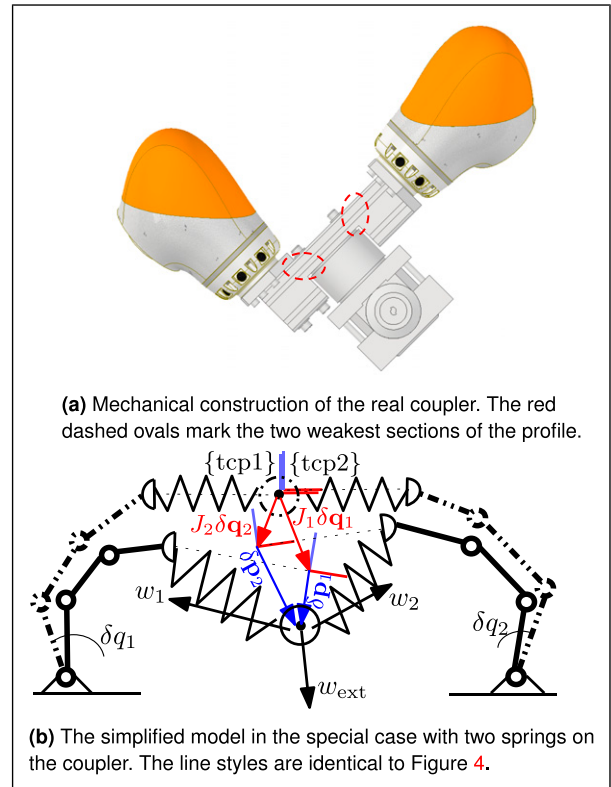


Figure 5. The illustrations of the simplified coupler elasticity for the derivation of the analytical stiffness expression.

whereby ${}^wR_{\text{tcp}} = {}^wR_{\text{tcp}}(\theta(u))$ is the rotation matrix between the two frames. Therefore, K_{tcp} is only dependent on the configuration $\theta(u)$, the constant stiffness of the robotic joints K_q and that of the coupler ${}^bK_{c,r}$ ($r = 1, \dots, n_{\text{rob}}$), but is not influenced by the internal preload tension. With a given w_{ext} and K_{tcp} , one obtains the TCP-deviation $\delta p_{\text{tcp}} = K_{\text{tcp}}^{-1} w_{\text{ext}}$. It is the pose difference between the cases with and without w_{ext} , but it does not represent the deviation from the path set-point $s(u)$.

If the coupler is rigid, $K_{c,r}^{-1} = O, \forall r = 1, \dots, n_{\text{rob}}$. Equation (8) is reduced to the additivity expression of individual robot stiffness matrices:

$$K_{\text{tcp}} = \sum_{r=1}^{n_{\text{rob}}} K_{\text{rob},r} = \sum_{r=1}^{n_{\text{rob}}} \left(J_r K_q^{-1} J_r^T \right)^{-1}. \quad (9)$$

The resulting stiffness is always higher than the case with a non-rigid coupler.

Formulation of the optimization problem

The planning problem is formulated mathematically as a constrained optimization problem in this section based upon the model derived in the last section.

The objective of the planning in the present paper is to enhance the capability K_{tcp} against w_{ext} along all the path segments. An objective function $\mathcal{J} \in \mathbb{R}$ is added to Problem 2 to formulate the following problem:

Problem 3. (Optimized Path-Constrained Configuration Planning)

Given an inter-connected multi-segment path $s_1(\cdot), \dots, s_{n_{\text{seg}}}(\cdot)$, find feasible configuration space trajectory segments $\theta_1^*(\cdot), \dots, \theta_{n_{\text{seg}}}^*(\cdot)$, satisfying the intra-segment constraints (2)–(4) for $\forall m \in \{1, \dots, n_{\text{seg}}\}$, as well as the inter-segment constraints (5) and (6), such that the value of $\mathcal{J} \in \mathbb{R}$, represented in segments as $\mathcal{J}(\theta_1(\cdot), \dots, \theta_{n_{\text{seg}}}(\cdot)) = \sum_{m=1}^{n_{\text{seg}}} \mathcal{J}_m(\theta_m(\cdot))$, is optimized as: $\mathcal{J}(\theta_1^*(\cdot), \dots, \theta_{n_{\text{seg}}}^*(\cdot)) = \min \mathcal{J}$.

The remainder of this subsection undertakes necessary modifications to Problem 3 to mathematically formulate the optimization problem in a solvable and light-weight form.

Decision variables

In Problem 3, the configuration space trajectory $\theta_m(\cdot) = [q_m^T(\cdot), y_m^T]^T, m = 1, \dots, n_{\text{seg}}$ is taken as decision variables. We use a B-spline to approximate each joint trajectory in $q_m(\cdot)$ with finite number of parameters. For executing path segment m , we define a B-spline of the order κ and the control point number l_m . The latter can be set

adaptively among segments of different lengths. In the following formulation of B-spline trajectories, the segment index m is omitted for better readability. The B-spline is defined using the array of knots (Kunoth et al., 2018):

$$\begin{aligned} \gamma &= [\gamma_0, \dots, \gamma_{l+\kappa}]^T \in \mathbb{R}^{l+\kappa+1} \\ 0 &= \gamma_1 = \dots = \gamma_{\kappa+1} < \gamma_{\kappa+2} < \dots \\ \dots &< \gamma_l < \gamma_{l+1} = \dots = \gamma_{l+\kappa+1} = 1. \end{aligned}$$

The knots between 0 and 1 are chosen with equal distance.

The base function of a B-spline is represented by $g_{i,\kappa}(u)$ with i denoting the sequence, κ the order, and $u \in [0, 1]$. Given below is the recursive formulation of the base functions (Lyche et al., 2018):

$$\begin{aligned} g_{i,1}(u) &= \begin{cases} 1, & \text{if } \gamma_i \leq u < \gamma_{i+1} \\ 0, & \text{otherwise} \end{cases} \\ g_{i,\kappa}(u) &= \frac{u - \gamma_i}{\gamma_{i+\kappa} - \gamma_i} g_{i,\kappa-1}(u) + \frac{\gamma_{i+\kappa+1} - u}{\gamma_{i+\kappa+1} - \gamma_{i+1}} g_{i+1,\kappa-1}(u). \end{aligned}$$

In the latter equation, a term is set to zero if the denominator has value zero. The trajectory of each joint $j = 1, \dots, n_j$ is approximated with a B-spline which is determined by the vector of control points $x_j \in \mathbb{R}^l$. The joint angle $q_j \in \mathbb{R}$ can thus be represented as:

$$q_j(u, x_j) = g^T(u) x_j \quad (10)$$

$$\text{whereby } g(u) = [g_{1,\kappa}(u), \dots, g_{l,\kappa}(u)]^T. \quad (11)$$

The property of B-spline ensures the l^1 -norm $\|g(u)\|_1 = 1$. All control points $x_j, j = 1, \dots, n_j$ are concatenated in a vector $x \in \mathbb{R}^{n_j l}$. We denote $z = [x_1^T, \dots, x_{n_j}^T, y^T]^T \in \mathbb{R}^{n_j l + n_{\text{pr}} + n_{\text{pw}}} = \mathbb{R}^{n_z}$ as the collective control point vector that governs not only the joint trajectories, but also the placement of robots and the work-piece. Therefore, the configuration space trajectory is represented as follows:

$$\theta(u, z) = \begin{bmatrix} q(u, x) \\ y \end{bmatrix} = G(u) \begin{bmatrix} x \\ y \end{bmatrix} = G(u) z, \quad (12)$$

whereby $G(u) \in \mathbb{R}^{n_\theta \times n_z}$ is constructed from $\tilde{G}(u) \in \mathbb{R}^{n_j \times l n_j}$ and identity matrix $I \in \mathbb{R}^{(n_{\text{pr}} + n_{\text{pw}}) \times (n_{\text{pr}} + n_{\text{pw}})}$ as follows:

$$G(u) = \begin{bmatrix} \tilde{G}(u) & I \end{bmatrix} = \begin{bmatrix} g_\kappa^T(u) & & & \\ & \ddots & & \\ & & g_\kappa^T(u) & \\ & & & I \end{bmatrix}. \quad (13)$$

The optimization problem to be formulated can thus be parameterized into one with decision variables being z_m for each segment $m = 1, \dots, n_{\text{seg}}$.

Constraints

After the parameterization of decision variables into finite-dimensional vectors $\mathbf{z}_m$, $m = 1, \dots, n_{\text{seg}}$ by (12), Problem 3 has a semi-infinite nature with infinite dimensional equality constraints (2). These constraints can only be approximately satisfied (Martin and Bobrow, 1999), but there exists no exact solution. A discretization of (2) in l_m points for segment m is undertaken:

$$\mathbf{e}_{\text{pose}}(\boldsymbol{\theta}_m(u, \mathbf{z}_m), \mathbf{s}_m(u)) = \mathbf{0}, \quad \forall u = u_1, \dots, u_{l_m}. \quad (14)$$

The number of points is set to be l_m , because the gradient of the discretized $n_e = l_m \times (n_{\text{cp}} + n_{\text{path}})$ dimensional function $\mathbf{e}_{\text{pose}}$ w.r.t. decision variables $\mathbf{z}_m$ is surjective according to Kupfer (1996). This means that at least one solution of $\mathbf{z}_m$ exists that brings the discretized constraint functions exactly to a zero vector as in (14).

In order to take over (4) in the problem formulation after the parameterization with B-splines, the strong convex hull property of B-splines (Piegl and Tiller, 2012) is applied. The joint trajectory is contained in the convex hull of its control point polygon. As long as the control points are inside the bounds, the joint trajectory is guaranteed to be contained inside. Therefore, (4) gets more conservative by an approximation $\mathbf{q}_{\min} \leq \mathbf{x}_m \leq \mathbf{q}_{\max}$. Combined with (3), one obtains:

$$\boldsymbol{\theta}_{\min} \leq \mathbf{z}_m \leq \boldsymbol{\theta}_{\max}, \quad (15)$$

whereby the bounds of non-placement joints are duplicated l_m times inside $\boldsymbol{\theta}_{\min}$ and $\boldsymbol{\theta}_{\max}$ to match the dimension of $\mathbf{z}_m$.

The time derivatives of (4) up to any given order are less trivial to be taken into account. Although B-splines of order κ ensures the continuity of its $\kappa - 1$ -th differentiation w.r.t. u , the magnitude of time derivatives depends on the positioning of control points $\mathbf{x}_m$ and the profile of u along the time axis. With unknown time profile of u given only after the phase of trajectory planning, it is still necessary in the planning phase to adjust the positioning of control points to limit the joint velocities. This is done by adding penalty terms in the object function. One undesirable behavior in velocity and acceleration is due to singularities, which are commonly close to stiff configurations (Aginaga et al., 2012). Around singularities, an extremely high joint velocity might be needed to achieve a slow Cartesian velocity of TCP. The sum of kinetostatic conditioning indices (Lin et al., 2017; Ranjbaran et al., 1996) for all robots is utilized for this penalty:

$$\mathcal{J}_{\text{sing}, m}(\boldsymbol{\theta}(u, \mathbf{z}_m)) = \sum_{r=1}^{n_{\text{rob}}} \sqrt{\text{tr}(\mathbf{J}_r \mathbf{J}_r^T) \text{tr}((\mathbf{J}_r \mathbf{J}_r^T)^{-1})}. \quad (16)$$

The other undesirable behavior is the fluctuation of a trajectory between its neighboring control points, which might occur since the equality constraint (14) is not imposed between the discretized points. Therefore, the following penalty term inspired by Kabir et al. (2021) is used inside of each segment (thus omitting segment index m):

$$\mathcal{J}_{\text{flc}}(\mathbf{z}) = \sum_{j=1}^{n_j} \sum_{i=1}^{l-1} \left[\frac{1}{\dot{q}_{\max j}} \cdot \frac{x_{j,i+1} - x_{j,i}}{u_{i+1} - u_i} \right]^2. \quad (17)$$

The inter-segment constraints (5) remain without change in the problem formulation, while (6), according to the B-spline property of endpoint interpolation (Piegl and Tiller, 2012), is equivalent to:

$$\begin{aligned} x_{m,j,l_m} &= x_{m+1,j,1} \\ \forall m &= 1, 2, \dots, n_{\text{seg}} - 1, \forall j = 1, \dots, n_j. \end{aligned} \quad (18)$$

The first time derivatives of (6) can be re-defined, assuming that du/dt at $u = 1$ of segment m is equal to du/dt at $u = 0$ of segment $m + 1$. Using the property of B-splines being tangent to the control polygon at the endpoints (Hoschek and Lasser, 1993), we have:

$$\begin{aligned} \frac{x_{m,j,l} - x_{m,j,l-1}}{\gamma_{m,l+1} - \gamma_{m,l}} &= \frac{x_{m+1,j,2} - x_{m+1,j,1}}{\gamma_{m+1,k+2} - \gamma_{m+1,k+1}} \\ \forall m &= 1, 2, \dots, n_{\text{seg}} - 1, \forall j = 1, \dots, n_j. \end{aligned} \quad (19)$$

Higher derivative orders of (6) can be obtained similarly.

Remark 1. Any further constraints such as the collision avoidance can be implemented as penalty terms in the objective function.

Objective function

We assume that the process wrench $\mathbf{w}_{\text{ext},m}(u)$ along $u \in [0, 1]$ is a known function for all segments $m = 1, \dots, n_{\text{seg}}$. The objective is to enhance K_{tcp} against $\mathbf{w}_{\text{ext},m}(u)$. Therefore, the preminent term in the objective function in segment m is a stiffness index $\mathcal{J}_{\text{stf},m}(u)$ of the multi-robot system. Inspired by Chiu (1988), such a stiffness index can be expressed as:

$$\begin{aligned} &\mathcal{J}_{\text{stf},m}(\boldsymbol{\theta}(u, \mathbf{z}_m), \mathbf{w}_{\text{ext},m}(u)) \\ &= \begin{cases} 0, & \text{if } \mathbf{w}_{\text{ext},m}(u) = \mathbf{0} \\ \mathbf{w}_{\text{ext},m}^T(u) \cdot \mathbf{K}_{\text{tcp}}^{-1}(\boldsymbol{\theta}(u, \mathbf{z}_m)) \cdot \frac{\mathbf{w}_{\text{ext},m}(u)}{\|\mathbf{w}_{\text{ext},m}(u)\|}, & \text{otherwise.} \end{cases} \end{aligned} \quad (20)$$

This stiffness index is proportional to the magnitude of $\mathbf{w}_{\text{ext},m}(u)$. Stiffness $K_{\text{top}}(\boldsymbol{\theta}(u, \mathbf{z}_m))$ is obtained from (9) which is dependent on robots' Jacobian matrices and thus on the configuration $\boldsymbol{\theta}(u, \mathbf{z}_m)$.

Weighted by constants $w_{\text{stf}}, w_{\text{sing}}, w_{\text{flc}} \in \mathbb{R}$, and combined with (16) and (17), the objective function for path segment m is expressed as follows:

$$\begin{aligned} \mathcal{J}_m(\mathbf{z}_m, \mathbf{w}_{\text{ext},m}(\cdot)) &= w_{\text{flc}} \mathcal{J}_{\text{flc},m}(\mathbf{z}_m) \\ &+ \frac{1}{l_m} \sum_{i=1}^{l_m} w_{\text{stf}} \mathcal{J}_{\text{stf},m}(u_i, \mathbf{z}_m, \mathbf{w}_{\text{ext},m}(u_i)) \\ &+ \frac{1}{l_m} \sum_{i=1}^{l_m} w_{\text{sing}} \mathcal{J}_{\text{sing},m}(u_i, \mathbf{z}_m). \end{aligned} \quad (21)$$

In (21), $u \in [0, 1]$ is discretized at l_m points to facilitate the summations. The discretization $u_i, i = 1, \dots, l_m$ is done at the identical points as in (14).

Mathematical formulation

With the illustration above, Problem 3 can be formulated mathematically as the following optimization problem:

Problem 4. (Parameter Optimization Problem for Path-Constrained Configuration Planning)

Given an interconnected multi-segment path $\mathbf{s}_1(\cdot), \dots, \mathbf{s}_{n_{\text{seg}}}(\cdot)$ and the corresponding nominal process wrench $\mathbf{w}_{\text{ext},1}(\cdot), \dots, \mathbf{w}_{\text{ext},n_{\text{seg}}}(\cdot)$, find the solution to:

$$\begin{aligned} \min_{\mathbf{z}_1, \dots, \mathbf{z}_{n_{\text{seg}}}} \quad & \mathcal{J} = \sum_{m=1}^{n_{\text{seg}}} \mathcal{J}_m(\mathbf{z}_m, \mathbf{w}_{\text{ext},m}(\cdot)) \\ \text{s.t.} \quad & \text{pose constraints (14), } \forall m = 1, \dots, n_{\text{seg}} \\ & \text{configuration bounds (15), } \forall m = 1, \dots, n_{\text{seg}} \\ & \text{inter-segment continuity (5)(18)(19).} \end{aligned} \quad (22)$$

Such a formulation enables the utility of solution methods for constrained parameter optimization problems.

Solution method set

This section illustrates the solution to Problem 4 with the reduced Hessian optimization in core and the parallel computation. An overview of necessary steps is given, before the parallel computation and the ADMM are illustrated. Afterward, in each individual thread of the parallel computation, the reduced Hessian optimization method is used, whose convergence guarantee is provided.

Overview of the solution procedure

The algorithm to solve Problem 4 is shown in Algorithm 1. The solution procedure consists of the parallel generation of multiple initial solutions (*initGuess*), parallel optimization of every single segment of all initiated solutions (*optimizeSingleSegment*), coordinated inter-segment constraint error update (*coordinateSegments*), and pruning of unfavorable solutions (*pruneCandidates*). After convergence, one candidate of interconnected trajectory segments is chosen (*selectBestCandidate*). The residual inter-segment constraint error of this candidate is eliminated in the end (*eliminateSegmentResidue*) so that it can be executed on the real robots with a B-spline interpolator.

Algorithm 1 solveProblem

Require: $\boldsymbol{\theta}_{\min}, \boldsymbol{\theta}_{\max}, n_{\text{seed}}$
Require: $\mathbf{s}_1(\cdot), \dots, \mathbf{s}_{n_{\text{seg}}}(\cdot), \mathbf{w}_{\text{ext},1}(\cdot), \dots, \mathbf{w}_{\text{ext},n_{\text{seg}}}(\cdot)$
 $\mathcal{I}_s, \mathcal{Z} \leftarrow \text{initGuess}(\mathbf{s}_1(\cdot), \dots, \mathbf{s}_{n_{\text{seg}}}(\cdot), \boldsymbol{\theta}_{\min}, \boldsymbol{\theta}_{\max}, n_{\text{seed}})$
 $\triangleright \mathcal{I}_s$ stands for the index set of active candidates
 $\triangleright \mathcal{Z}$ stands for the set of guessed solutions
 $\Lambda \in \mathbb{R}^{|\mathcal{I}_s| \times (n_{\text{seg}}-1) \times n_{\text{cc}}} \leftarrow \mathbf{O} \quad \triangleright$ with n_{cc} being the number of continuity constraint equations in (5)(18)(19)
while not converged **do**
 for all $(\mathbf{z}_{i_s,m} \in \mathcal{Z}$ with subscript $i_s \in \mathcal{I}_s$ and $m = 1, \dots, n_{\text{seg}})$ **parallel do**
 $\mathbf{z}_{i_s,m}^{(\text{new})} \leftarrow \text{optimizeSingleSegment}(\mathbf{z}_{i_s,m}, \mathbf{s}_m(\cdot), \mathbf{w}_{\text{ext},m}(\cdot), \boldsymbol{\theta}_{\min}, \boldsymbol{\theta}_{\max}, \Lambda_{i_s})$
 Update $\mathcal{Z}$ with the entry $\mathbf{z}_{i_s,m}^{(\text{new})}$
 end for
 for $i_s \in \mathcal{I}_s$ **do**
 $\Lambda_{i_s} \leftarrow \text{coordinateSegments}(\mathbf{z}_{i_s,1}^{(\text{new})}, \dots, \mathbf{z}_{i_s,n_{\text{seg}}}^{(\text{new})})$
 end for
 $\mathcal{I}_s \leftarrow \text{pruneCandidates}(\mathcal{I}_s, \mathcal{Z}) \quad \triangleright$ less elements in $\mathcal{I}_s$
end while
 $\mathbf{z}_1, \dots, \mathbf{z}_{n_{\text{seg}}} \leftarrow \text{selectBestCandidate}(\mathcal{I}_s, \mathcal{Z})$
 $\mathbf{z}_1^*, \dots, \mathbf{z}_{n_{\text{seg}}}^* \leftarrow \text{eliminateSegmentResidue}(\mathbf{z}_1, \dots, \mathbf{z}_{n_{\text{seg}}})$
return $\mathbf{z}_1^*, \dots, \mathbf{z}_{n_{\text{seg}}}^*$

In such a solution procedure, multiple local optima of the trajectory planning problem can be tackled efficiently with multiple initial guesses being generated, optimized, compared, and pruned in parallel computation. For a complex multi-segment path, the decentralized optimization in segments has a better scalability w.r.t. path length than solving for the whole trajectories. Moreover, due to the complexity of the path, initial guesses either by random sampling or by manual specification of robot-workpiece placement are unlikely to be feasible in terms of whole-path reachability within bounds of joint angles. Infeasible initial guesses are nevertheless accepted, because the joint configurations at the starting and finishing path points as well as the placements are all decision variables that can be adjusted in the later optimization process. If

some placement dimensions must be fixed at a value, one provides an equal upper and lower bound. After convergence of the optimization, the only post-processing is the elimination of a slight amount of residual configuration jumps between segments (*eliminateSegmentResidue*).

Parallelization with multiple initial solutions

Inspired by Matlab multi-start and global search algorithms (Ugray et al., 2007), multiple initial solutions are generated in parallel. In the generation of initial solutions, as shown in Algorithm 2, configurations corresponding to the starting point on the first segment of the path $s_1(0)$ are determined by analytical inverse kinematics (AIK) (Shimizu et al., 2008). The corresponding function *aInvKin* only accepts a single 6-axis robot model, so any redundant joint $q_{red,0}$ of each individual robot as well as its further DOFs in the placement y_0 must be determined beforehand from function *getRandom*. For each robot $r = 1, \dots, n_{rob}$, AIK outputs at most eight solutions, which lead to up to $8^{n_{rob}}$ combinations for all robots by function *combiConfig*.

After collecting all sampled configurations Θ for the starting point of the first path segment, the following computations are independent from each other and can be parallelized for different starting points $\theta_{1,1} \in \Theta$. By solving the iterative inverse kinematics using function *invKin* with the solution of the last point $\theta_{m,i-1}$ as an input, the way-points of the configuration space trajectories are created. If *invKin* finds no reachable solution within joint limits, the solution of the last way-point replaces the current one. The resulting infeasibility can be corrected during the optimization later, because the placements of the robots and the work-piece are decision variables that are adjusted along with the configuration space trajectories until the reachability is satisfied. An alternative is to exploit this correction of infeasible solutions by setting all way-points identical to the starting point $\theta_{1,1}$. In this case, the more time-consuming function *invKin* in Algorithm 2 is replaced by $\theta_{m,i} \leftarrow \emptyset$.

After obtaining the way-points of the configuration space trajectories, an approximation by B-splines with the function *approximateSpline* is conducted, creating the collective control point vector $z_{k_{seed},m}$ that is stored in $\mathcal{Z}$. The seed index set $\mathcal{I}_s = \{1, \dots, k_{seed}\}$ with k_{seed} being the cardinality of Θ is also returned for use in Algorithm 1.

Algorithm 2 *initGuess*

Require: $s_1(\cdot), \dots, s_{n_{seg}}(\cdot), \theta_{min}, \theta_{max}, n_{seed}$
 $\Theta = \emptyset$ $\triangleright$ configurations for the starting point on path
for $j = 1, \dots, n_{seed}$ **do**
 $y_0, q_{red,0} \leftarrow getRandom(\theta_{min}, \theta_{max})$
for $r = 1, \dots, n_{rob}$ **do**
 $\{\theta_{1,1}^{(1)}, \dots, \theta_{1,1}^{(8)}\}_r \leftarrow aInvKin(s_1(0), y_0, q_{red,0}, r)$
end for
 $\Theta_j \leftarrow combiConfig(\{\theta_{1,1}^{(1)}, \dots, \theta_{1,1}^{(8)}\}_{1:n_{rob}}, y_0, q_{red,0})$
 $\Theta \leftarrow \Theta \cup \Theta_j$
end for
 $\mathcal{Z} = \emptyset$ $\triangleright$ storage of decision variables for all starting configurations and all segments
 $k_{seed} = 0$
for all $\theta_{1,1} \in \Theta$ **parallel do**
 $k_{seed} \leftarrow k_{seed} + 1$
for $m = 1, \dots, n_{seg}$ **do**
for $i = 2, \dots, l_m$ **do**
 $\theta_{m,i} = invKin(s_m(u_i), y_0, \theta_{m,i-1}, \theta_{min}, \theta_{max})$
if $\theta_{m,i} = \emptyset$ **then** $\triangleright$ infeasible solution
 $\theta_{m,i} \leftarrow \theta_{m,i-1}$
end if
end for
 $z_{k_{seed},m} \leftarrow approximateSpline(\theta_{m,1}, \dots, \theta_{m,l_m})$
 $\mathcal{Z} \leftarrow \mathcal{Z} \cup \{z_{k_{seed},m}\}$
if $m < n_{seg}$ **then**
 $\theta_{m+1,1} \leftarrow \theta_{m,l_m}$ $\triangleright$ initialize the first point of segment $m+1$ by the last point of segment m
end if
end for
return $\mathcal{I}_s = \{1, \dots, k_{seed}\}$ $\triangleright$ index of active candidates
 $\mathcal{Z} = \{z_{i_s,m} \in \mathbb{R}^{n_z} | i_s = 1, \dots, k_{seed}; m = 1, \dots, n_{seg}\}$

The initial solutions are optimized in parallel in Algorithm 1. After each step of optimization, *pruneCandidates* reduces the number of further considered solutions. The solutions that remain in the active set $\mathcal{I}_s$ are further optimized. The last comparison of object function values is carried out by *selectBestCandidate* to find the global optimum.

Inter-segment coordinated descend using ADMM

Simultaneous to the parallel optimization of multiple initial solutions, the segments of Problem 4 are decentrally optimized in a parallel manner in Algorithm 1, so that the scalability w.r.t. path length is enhanced.

For the decentralization among segments, methods such as ADMM (Boyd et al., 2011) and optimality condition decomposition (OCD) (Conejo et al., 2006) are considered. With OCD, equality constraints (5), (18), and (19) partially remain in subproblems. In Algorithm 1, *optimizeSingleSegment* may have difficulties handling these inter-segment equality constraints with the pose constraints (14) together. On the contrary, ADMM relaxes these equality constraints into the objective function (Wang et al., 2017), allowing *optimizeSingleSegment* fully exploit the kinematic structure of equality constraints (14).

The residual constraint error due to the relaxation can be eliminated in the end.

Between adjacent path segments $m = 1, \dots, n_{\text{seg}}$ of one active solution candidate $i_s \in \mathcal{I}_s$, there are $n_{\text{seg}} - 1$ connections. For each connection, the continuity constraints (5), (18), and (19) are of dimension n_{cc} . Therefore, in Algorithm 1, the dual variables $\Lambda_{i_s} \in \mathbb{R}^{(n_{\text{seg}}-1) \times n_{\text{cc}}}$ are defined and set to zero initially, whereas Λ collects all dual variables for all solution candidates in $\mathcal{I}_s$. Constraints (5), (18), and (19) can be combined into the matrix form:

$$C_{m,1}\mathbf{z}_m - C_{m+1,0}\mathbf{z}_{m+1} = \mathbf{0}, \forall m = 1, \dots, n_{\text{seg}} - 1,$$

whereby $C_{m,0}, C_{m,1} \in \mathbb{R}^{n_{\text{cc}} \times n_{z_m}}$, $m = 1, \dots, n_{\text{seg}}$ are constant matrices with the subscript 0 standing for the extraction of control points from the segment start, and the subscript 1 for the segment finish.

Therefore, Problem 4 can be decentralized into n_{seg} subproblems with the augmented Lagrangian. Accordingly, the function *optimizeSingleSegment* in Algorithm 1 is mathematically formulated as follows:

$$\begin{aligned} \min_{\mathbf{z}_m} \quad & \tilde{\mathcal{J}}_m = \mathcal{J}_m(\mathbf{z}_m, \mathbf{w}_{\text{ext},m}(\cdot)) \\ & + \frac{1}{2} \|C_{m-1,1}\tilde{\mathbf{z}}_{m-1} - C_{m,0}\mathbf{z}_m + \boldsymbol{\lambda}_{m-1}\|_P^2 \\ & + \frac{1}{2} \|C_{m,1}\mathbf{z}_m - C_{m+1,0}\tilde{\mathbf{z}}_{m+1} + \boldsymbol{\lambda}_m\|_P^2 \\ \text{s.t.} \quad & \text{pose constraints (14)} \\ & \text{configuration bounds (15),} \end{aligned} \quad (23)$$

whereby the symbol $\|\mathbf{a}\|_P^2$ for any vector $\mathbf{a}$ represents the operation $\mathbf{a}^T \mathbf{P} \mathbf{a}$ with $\mathbf{P}$ being a diagonal matrix with constant coefficients. The vector of dual variables $\boldsymbol{\lambda}_m \in \mathbb{R}^{n_{\text{cc}}}$ is the transpose of the m -th row of Λ_{i_s} , with $i_s \in \mathcal{I}_s$ being the index of the active solution candidate. Vectors $\tilde{\mathbf{z}}_{m-1}$ and $\tilde{\mathbf{z}}_{m+1}$ are the last solutions of $\mathbf{z}_{m-1}$ and $\mathbf{z}_{m+1}$ which are updated from other subproblems. For subproblems of segment $m = 1$ and $m = n_{\text{seg}}$ without a neighboring segment on one side, the second or the third term in (23) vanishes, respectively.

In Algorithm 1, *optimizeSingleSegment* solves the subproblem (23) and returns $\mathbf{z}_{i_s,m}^{(\text{new})}$ which is then used to update the dual variables in *coordinateSegments* as follows:

$$\begin{aligned} \boldsymbol{\lambda}_m^{(\text{new})} & \leftarrow \boldsymbol{\lambda}_m + C_{m,1}\mathbf{z}_m^{(\text{new})} - C_{m+1,0}\mathbf{z}_{m+1}^{(\text{new})}, \\ \forall m & = 1, \dots, n_{\text{seg}} - 1. \end{aligned}$$

Afterward, Λ_{i_s} is updated with these entries.

After the convergence and the selection of a single best candidate, function *eliminateSegmentResidue* eliminates the tiny amount of residual error of inter-segment continuity (5), (18), and (19) as follows. The average values among decision variables across neighboring segments are taken as the inter-segment target of correction. These include the

average of placement results, as well as the average between the last and the first control point of two B-spline corresponding to s_m (1) and s_{m+1} (0) on path. The affected decision variables $\mathbf{z}_1, \dots, \mathbf{z}_{n_{\text{seg}}}$ are corrected to the inter-segment target individually, resulting in $\bar{\mathbf{z}}_1, \dots, \bar{\mathbf{z}}_{n_{\text{seg}}}$. Such individual adjustments cause violations of intra-segment constraints (14), resulting in $\mathbf{e}_{\text{pose}}(\boldsymbol{\theta}_m(u, \bar{\mathbf{z}}_m), s_m(u)) \neq \mathbf{0}$. To restore the intra-segment constraints, the necessary adjustment of all joint configurations $\delta\boldsymbol{\theta}_m(u)$ at any $u \in [0, 1]$ can be obtained with the help of the pseudo inverse of the Jacobian $\nabla_{\boldsymbol{\theta}} \mathbf{e}_{\text{pose}}(u, \mathbf{z}_m) \in \mathbb{R}^{(n_J+n_{\text{pr}}+n_{\text{pw}}) \times (n_{\text{cp}}+n_{\text{path}})}$:

$$\delta\boldsymbol{\theta}_m(u) = (\nabla_{\boldsymbol{\theta}} \mathbf{e}_{\text{pose}}(u, \mathbf{z}_m))^T + \mathbf{e}_{\text{pose}}(\boldsymbol{\theta}_m(u, \bar{\mathbf{z}}_m), s_m(u)).$$

Using the adjusted joint configurations, the decision variables are computed again by B-spline approximation according to the relation (12) so as to generate $\mathbf{z}_1^*, \dots, \mathbf{z}_{n_{\text{seg}}}^*$. This result should be close to the values before executing *eliminateSegmentResidue* because the residual error of inter-segment continuity is already tiny by ADMM.

Solving optimization problem with reduced Hessian method

The reduced Hessian method is utilized to solve each single subproblem (23) for each of the individual segments. This corresponds to the function *optimizeSingleSegment* in Algorithm 1. So in the current section, segment index m is sometimes neglected for better readability. Subproblem (23) is solved in a Sequential Quadratic Programming (SQP) manner iteratively as follows in the k -th iteration:

$$\min_{\mathbf{d} \in \mathbb{R}^{n_z}} \quad \nabla_{\mathbf{z}} \tilde{\mathcal{J}}_m(\mathbf{z}_k) \mathbf{d} + \frac{1}{2} \mathbf{d}^T \mathbf{B}_k \mathbf{d} \quad (24a)$$

$$\text{s.t.} \quad \mathbf{e}_{\text{pose}}(\mathbf{z}_k) + \nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}(\mathbf{z}_k) \mathbf{d} = \mathbf{0}, \quad (24b)$$

$$\boldsymbol{\theta}_{\min} \leq \mathbf{z}_k + \mathbf{d} \leq \boldsymbol{\theta}_{\max}, \quad (24c)$$

whereby $\mathbf{e}_{\text{pose}}(\mathbf{z}_k) \in \mathbb{R}^{n_e}$ is the concatenation of $\mathbf{e}_{\text{pose}}(u, \mathbf{z}_k)$, $\forall u = u_1, \dots, u_{l_m}$ from (14). The gradient of $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ w.r.t. $\mathbf{z}_k$ is represented by $\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}} \in \mathbb{R}^{n_z \times n_e}$. The gradient of the objective function in the optimization problem (23) is $\nabla_{\mathbf{z}} \tilde{\mathcal{J}}_m \in \mathbb{R}^{n_z}$ which has a dependency on $\boldsymbol{\lambda}_{m-1}$ and $\boldsymbol{\lambda}_m$. The matrix $\mathbf{B}_k \in \mathbb{R}^{n_z \times n_z}$ represents the Hessian. Problem (24a, b, c) is an optimization w.r.t. the descent step $\mathbf{d} \in \mathbb{R}^{n_z}$, representing the adjustment of the original decision variables $\mathbf{z}_k$. In the reduced Hessian setting, $\mathbf{d}$ consists of a vertical and a horizontal step, expressed as follows:

$$\mathbf{d} = \mathbf{v} + \alpha \mathbf{h}, \quad (25)$$

whereby the vertical step $\mathbf{v}$ is a solution of (24b) and eliminates the pose error $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, so as to maintain the path tracking and coupling:

$$\mathbf{e}_{\text{pose}}(\mathbf{z}_k) + \nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k) \mathbf{v} = \mathbf{0}, \quad (26)$$

whereas the horizontal step $\mathbf{h}$, with $0 < \alpha \leq 1$ as the step length in the line search, enables the descent of the objective function (23) that improves the capabilities of robots without changing the values of $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$. This can be achieved if zero matrix O is obtained from:

$$\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k) \mathbf{h} = O. \quad (27)$$

This means that the horizontal step does not vanish only in the null-space of $\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, so a null-space basis $Z_z(\mathbf{z}_k) \in \mathbb{R}^{n_z \times (n_z - n_e)}$ is to be constructed that satisfies:

$$\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k) Z_z(\mathbf{z}_k) = O, \quad (28)$$

$$Z_z(\mathbf{z}_k) Z_z(\mathbf{z}_k)^+ + \left(\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T \right)^+ \nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T = I. \quad (29)$$

Equation (28) guarantees that all column vectors of $Z_z(\mathbf{z}_k)$ lie inside the null-space of $\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ while (29) guarantees that $Z_z(\mathbf{z}_k)$ spans the complete null-space of $\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ without losing ranks. Any type of pseudo-inverse can be applied in (29), here denoted with the superscript $+$. The horizontal descending step $\mathbf{h}$ is a linear combination of columns of $Z_z(\mathbf{z}_k)$:

$$\mathbf{h} = Z_z(\mathbf{z}_k) \bar{\mathbf{d}}. \quad (30)$$

whereby $\bar{\mathbf{d}} \in \mathbb{R}^{n_z - n_e}$ represents the null-space descent. The null-space basis $Z_z(\mathbf{z}_k)$ maps the null-space descent into the space of all decision variables. One may substitute (25) and (30) into (24a, b, c), and take $\bar{\mathbf{d}}$ as the new decision variable, so as to reformulate the problem as follows:

$$\min_{\bar{\mathbf{d}}} \quad \nabla_{\mathbf{z}} \tilde{\mathcal{J}}_m^T Z_z(\mathbf{z}_k) \bar{\mathbf{d}} + \mathbf{v}^T B_k Z_z(\mathbf{z}_k) \bar{\mathbf{d}} + \frac{1}{2} \bar{\mathbf{d}}^T \bar{B}_k \bar{\mathbf{d}} \quad (31a)$$

$$\text{s.t.} \quad \begin{bmatrix} Z_z(\mathbf{z}_k) \\ -Z_z(\mathbf{z}_k) \end{bmatrix} \bar{\mathbf{d}} \leq \begin{bmatrix} \boldsymbol{\theta}_{\max} - \mathbf{z}_k - \mathbf{v} \\ -(\boldsymbol{\theta}_{\min} - \mathbf{z}_k - \mathbf{v}) \end{bmatrix}, \quad (31b)$$

whereby $\bar{B}_k = Z_z(\mathbf{z}_k)^T B_k Z_z(\mathbf{z}_k)$ is the reduced Hessian matrix of a dimension $\mathbb{R}^{(n_z - n_e) \times (n_z - n_e)}$. The cross term $\mathbf{v}^T B_k Z_z(\mathbf{z}_k) \bar{\mathbf{d}}$ can be ignored as long as $\mathbf{v}$ in this term converges more quickly to zero than $\bar{\mathbf{d}}$ does (Schmid and Biegler, 1994). If $Z_z(\mathbf{z}_k)$ and $\nabla_{\mathbf{z}} \tilde{\mathcal{J}}(\mathbf{z}_k)$ are provided, problem (31a, b) becomes solvable by updating $\bar{B}_k$ with the Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm (Nocedal and Wright, 1999).

After solving both $\mathbf{v}$ and $\mathbf{h}$ for (25), an iteration is made with $\mathbf{z}_{k+1} = \mathbf{z}_k + \mathbf{d}(\mathbf{z}_k, \bar{B}_k)$.

The following subsections are devoted to the construction of the necessary variables $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, $\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, $Z_z(\mathbf{z}_k)$,

and $\nabla_{\mathbf{z}} \tilde{\mathcal{J}}_m$, which guarantees the convergence of the reduced Hessian method.

Vertical step. To obtain $\mathbf{v}$, one needs to determine $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ and $\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, before solving the system of equation (26). The vector $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ represents the pose difference between frames. Its components are shown in Figure 3. For the physical coupling between robot i and j at step $u \in [0, 1]$, this pose difference is calculated between the homogeneous transformation matrices of $\{\text{tcp}_i\}$ and $\{\text{tcp}_j\}$, denoted as $\text{diff}(T_i(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k), T_j(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k))$. For the path tracking by robot i , the pose difference is between the homogeneous transformation matrices of the way-point on the path (T_{wpt}) and $\{\text{tcp}_i\}$, denoted as $\text{diff}(T_{\text{wpt}}(\mathbf{s}(u), \mathbf{y}_k), T_i(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k))$. The operator $\text{diff}(\cdot, \cdot)$ is defined as follows:

$$\text{diff}(T_1, T_2) = \begin{bmatrix} R_2 \cdot \Phi((R_2)^T R_1) \\ \mathbf{t}_2 - \mathbf{t}_1 \end{bmatrix}, \quad (32)$$

whereby the rotation matrices R and the translation vectors $\mathbf{t}$ are extracted from the homogeneous transformation matrix $T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$. The conversion from rotation matrix to Euler angle is denoted as $\Phi(\cdot)$.

W.l.o.g., the complete pose error $\mathbf{e}_{\text{pose}}$ at a given $u \in [0, 1]$ consists of the following errors:

$$\mathbf{e}_{\text{pose}}(u, \mathbf{z}_k) = \begin{bmatrix} \text{diff}(T_{\text{wpt}}(\mathbf{s}(u), \mathbf{y}_k), T_1(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k)) \\ \text{diff}(T_2(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k), T_1(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k)) \\ \dots \\ \text{diff}(T_{n_{\text{rob}}}(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k), T_1(\mathbf{q}(u, \mathbf{x}_k), \mathbf{y}_k)) \end{bmatrix}. \quad (33)$$

Finally, $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ is the concatenation of $u = u_1, \dots, u_{l_m}$ in increasing sequence with $u_1 = 0$ and $u_{l_m} = 1$:

$$\mathbf{e}_{\text{pose}}(\mathbf{z}_k) = \begin{bmatrix} \mathbf{e}_{\text{pose}}(u_1, \mathbf{z}_k) \\ \vdots \\ \mathbf{e}_{\text{pose}}(u_{l_m}, \mathbf{z}_k) \end{bmatrix}.$$

The synchronization between the path $\mathbf{s}(u)$ and the joint trajectories $\mathbf{q}(u, \mathbf{z}_k)$ must be ensured. This means, at any $u \in [0, 1]$, $\mathbf{s}(u)$ and $\mathbf{q}(u, \mathbf{z}_k)$ should progress equally. If, for example, $\mathbf{q}(u, \mathbf{z}_k)$ progresses faster than $\mathbf{s}(u)$ at start as shown in Figure 6, $\mathbf{q}(u, \mathbf{z}_k)$ might move more than necessary (usually in a twisting manner) between u_1 and u_2 , before being forced by the vertical step update to meet with the slow moving $\mathbf{s}(u)$ at point u_2 , leading to fluctuations.

To achieve the synchronization, $\mathbf{s}(u)$ is approximated by a B-spline $\mathbf{s}'(u, \mathbf{z}_s)$ in Cartesian space with the same order and control point number as the joint space B-splines $\mathbf{q}(u, \mathbf{x})$. This replacement brings a cost of less accurate path tracking on the way-points due to the approximation error

of the B-spline, but we get rid of the phenomenon in Figure 6.

The next step is to determine $\nabla_z \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$. As shown in Figure 7, two mappings are incorporated in $\nabla_z \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, one between the Cartesian space adjustments to eliminate $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ and the configuration space adjustments $\Delta\theta(u, \mathbf{z}_k)$ on way-points $u_i, i = 1, \dots, l_m$, the other between $\Delta\theta(u, \mathbf{z}_k)$ and the adjustment of control points $\mathbf{z}_k$, that is, the vertical step $\mathbf{v}$.

The first mapping is done by the Jacobian:

$$\nabla_{\theta} \mathbf{e}_{\text{pose}}(u, \mathbf{z}) = \begin{bmatrix} J_{\text{wpt}}(\mathbf{s}(u), \mathbf{y}) - J_1(\mathbf{q}(u, \mathbf{x}), \mathbf{y}) \\ J_1(\mathbf{q}(u, \mathbf{x}), \mathbf{y}) - J_2(\mathbf{q}(u, \mathbf{x}), \mathbf{y}) \\ \dots \\ J_1(\mathbf{q}(u, \mathbf{x}), \mathbf{y}) - J_{n_{\text{rob}}}(\mathbf{q}(u, \mathbf{x}), \mathbf{y}) \end{bmatrix}^T, \quad (34)$$

whereas the second mapping is made with the help of the basis functions $G(u)$ from (13) of B-splines:

$$\nabla_z \mathbf{e}_{\text{pose}}(u, \mathbf{z}_k) = G^T(u) \nabla_{\theta} \mathbf{e}_{\text{pose}}(u, \mathbf{z}_k). \quad (35)$$

Finally, by concatenating $u = u_1, \dots, u_{l_m}$, one gets:

$$\nabla_z \mathbf{e}_{\text{pose}}(\mathbf{z}_k) = [\nabla_z \mathbf{e}_{\text{pose}}(u_1, \mathbf{z}_k), \dots, \nabla_z \mathbf{e}_{\text{pose}}(u_{l_m}, \mathbf{z}_k)]. \quad (36)$$

After obtaining $\mathbf{e}_{\text{pose}}(\mathbf{z}_k)$ and $\nabla_z \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, the solution of (26) follows that for an under-determined linear system of equations. One may solve (26) with Moore-Penrose inverse that satisfies the equation exactly while minimizing $\|\mathbf{v}\|$:

$$\begin{aligned} \mathbf{v} &= -\left(\nabla_z \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k)\right)^+ \mathbf{e}_{\text{pose}}(\mathbf{z}_k) \\ &= -\nabla_z \mathbf{e}_{\text{pose}} \left(\nabla_z \mathbf{e}_{\text{pose}}^T \nabla_z \mathbf{e}_{\text{pose}} \right)^{-1} \mathbf{e}_{\text{pose}}(\mathbf{z}_k). \end{aligned} \quad (37)$$

Remark 2. If $\nabla_z \mathbf{e}_{\text{pose}}(\mathbf{z})$ has high condition number in case of using high-order basis functions in $G(u)$, $\|\mathbf{v}\|$ can still be large, causing strong fluctuation in the resulting configuration space trajectories. A trade-off between the accuracy of the equality in (26) and the stability of $\mathbf{v}$ is achieved by solving an over-determined linear system of

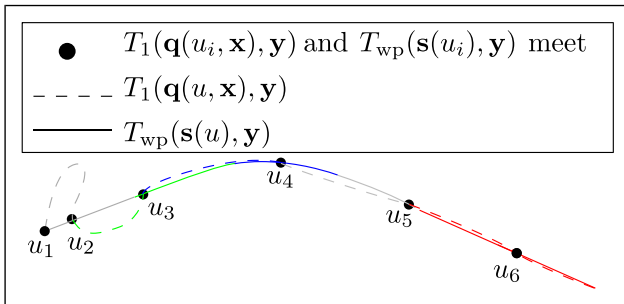


Figure 6. Asynchronization between $\mathbf{s}(u)$ and $\mathbf{q}(u, \mathbf{z}_k)$, $u \in [0, 1]$ results in the fluctuation of the robot movement. The path $\mathbf{s}(u)$ and the joint trajectory $\mathbf{q}(u, \mathbf{z}_k)$ are visualized here in the Cartesian space after transformations by the forward kinematics.

equations using the regularization method (Neumaier, 1998):

$$\begin{aligned} \tilde{\mathbf{v}} &= -\begin{bmatrix} \nabla_z \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k) \\ \varepsilon I \end{bmatrix}^+ \begin{bmatrix} \mathbf{e}_{\text{pose}}(\mathbf{z}_k) \\ O \end{bmatrix} \\ &= -\left(\begin{bmatrix} \nabla_z \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k) \\ \varepsilon I \end{bmatrix}^T \begin{bmatrix} \nabla_z \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k) \\ \varepsilon I \end{bmatrix} \right)^{-1} \\ &\quad \cdot \begin{bmatrix} \nabla_z \mathbf{e}_{\text{pose}}^T(\mathbf{z}_k) \\ \varepsilon I \end{bmatrix}^T \begin{bmatrix} \mathbf{e}_{\text{pose}}(\mathbf{z}_k) \\ O \end{bmatrix} \\ &\quad (0 < \varepsilon \ll 1). \end{aligned} \quad (38)$$

Null-space basis. We present a method to find $Z_z(\mathbf{z}_k) \in \mathbb{R}^{n_z \times (n_z - n_e)}$ that not only satisfies (28), but is also kinematically interpretable and continuous w.r.t. changes of $\mathbf{z}_k$. Such a new robotics-specific null-space basis is necessary, because the commonly used decomposition methods, for example, orthonormal null-space basis obtained from the singular value decomposition or QR decomposition of $\nabla_z \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$, guarantee no continuity w.r.t. $\mathbf{z}_k$ and thus no convergence with projected quasi-Newton algorithms such as BFGS can be proved (Coleman and Sorensen, 1984). On the contrary, our $Z_z(\mathbf{z}_k)$ allows the provable convergence of the reduced Hessian method.

To build such $Z_z(\mathbf{z}_k)$, we interpret each column as the necessary adjustment of the entire $\mathbf{z}_k$ to achieve a single-point adjustment of a redundant configuration without changing the pose error. On the one hand, redundant configurations include, if a robot possesses more than six joints, those redundant joints of $\theta(u, \mathbf{z}_k) = [\mathbf{q}(u, \mathbf{x}_k)^T, \mathbf{y}_k^T]^T$ that do not occur in a conventional 6-axis robot. Each of these redundant joints makes up l_m single-point adjustments, because it can be adjusted individually at l_m sampled points along the discretized path. On the other hand, each placement variable $\mathbf{y}$ that occurs in $\theta(u, \mathbf{z}_k)$ is regarded as one single-point adjustment of redundant configuration. There are totally $n_z - n_e$ possible single-point adjustments of redundant configurations, corresponding to the columns of Z_z . Their linear independence ensures the full column rank of Z_z and thus (29).

The single-point adjustment of a redundant configuration is denoted as $\delta\theta_{\text{red},j} \in \mathbb{R}^{n_\theta}$, with only the entry of the redundant configuration being one and all others zero. The value of j ranges from one to the total number

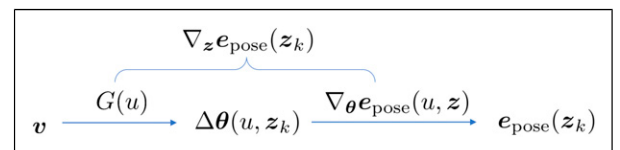


Figure 7. The two mappings incorporated in $\nabla_z \mathbf{e}_{\text{pose}}(\mathbf{z}_k)$.

of redundant configurations $n_{\text{red},\theta} = n_\theta - n_{\text{cp}} - n_{\text{path}}$. If we denote $n_{\text{red,joint}} = n_\theta - n_{\text{cp}} - n_{\text{path}} - n_{\text{pr}} - n_{\text{pw}}$, then $j = 1, \dots, n_{\text{red,joint}}$ corresponds to the redundant joints and $j = n_{\text{red,joint}} + 1, \dots, n_{\text{red},\theta}$ the placement variables.

Afterward, one should determine the corresponding error-free single-point reactive adjustment of the whole configuration (including non-redundant joints) $\delta\theta_{N,j}(u, z_k) \in \mathbb{R}^{n_\theta}$ that neither changes the pose error nor the other redundant configurations. For example, when the redundant joint of a robot is adjusted, only the joint configuration of this single robot is allowed to react, but the placements do not. Generally, one can use the result from (34) to build the error Jacobian $J_e = \nabla_{\theta} e_{\text{pose}}^T(u, z)$. To make the error-free single-point reactive adjustment of the whole configuration have no influence on other redundant configurations, one can set the columns of J_e that correspond to other redundant configurations to zero. Afterward, one can construct a projector as follows:

$$N_j(\theta(u, z_k)) = I - (J_e(u, z_k))^+ J_e(u, z_k). \quad (39)$$

By projecting the single-point adjustment of the redundant configuration, the error-free adjustment of the whole configuration is obtained:

$$\delta\theta_{N,j}(u, z_k) = N_j(\theta(u, z_k)) \delta\theta_{\text{red},j}. \quad (40)$$

Lemma 1. *The error-free adjustment of the whole configuration $\delta\theta_{N,j}(u, z_k)$ induces no change to the path and coupling errors, that is:*

$$\nabla_{\theta} e_{\text{pose}}^T \delta\theta_{N,j}(u, z_k) = \mathbf{0}.$$

Proof. See Appendix A.

Finally, the entire z should be adjusted according to the mapping of $G(u)$ (12) to enable this error-free adjustment. A case distinction is made here. When a column of $Z_z(z_k)$ corresponds to the adjustment $\delta z_{\text{wpt},j}(u)$ that aims at changing the angle of a redundant joint at one of the l_m sampled waypoints along the discretized path, this column is computed as:

$$\delta z_{\text{wpt},j}(u, z_k) = G(u)^+ \delta\theta_{N,j}(u, z_k), j = 1, \dots, n_{\text{red,joint}}. \quad (41)$$

when a column of $Z_z(z_k)$ corresponds to the adjustment $\delta z_{\text{plc},j}$ that aims at changing the placement variable, it is computed as follows, which considers all the adjustments along the discretized path but results in a single adjustment of a static placement variable:

$$\delta z_{\text{plc},j}(z_k) = \begin{bmatrix} G(u_1) \\ \vdots \\ G(u_{l_m}) \end{bmatrix}^+ \begin{bmatrix} \delta\theta_{N,j}(u_1, z_k) \\ \vdots \\ \delta\theta_{N,j}(u_{l_m}, z_k) \end{bmatrix}, \quad (42)$$

$$j = n_{\text{red,joint}} + 1, \dots, n_{\text{red},\theta}.$$

Remark 3. *In case of ill-conditioning of the pseudo inverse in (41) and (42), the regularization technique as in Remark 2 can be applied.*

The above steps that generate the columns of the null-space basis are summarized in Algorithm 3.

Algorithm 3 Generation of the null-space basis

```

 $Z_z \leftarrow O^{n_z \times (n_z - n_e)}$ 
for  $i = 1, \dots, l_m$  do
  for  $j = 1, \dots, n_{\text{red},\theta}$  do
    Obtain  $J_{e,j}(\theta(u_i, z))$ ,  $\delta\theta_{\text{red},j}$ 
    Store  $\delta\theta_{N,j}(u_i, z)$  obtained from (39)(40)
  end for
end for
 $n_{\text{cnt}} \leftarrow 1$ 
for  $j = 1, \dots, n_{\text{red,joint}}$  do
  for  $i = 1, \dots, l_m$  do
     $Z_z(:, n_{\text{cnt}}) \leftarrow \delta z_{\text{wpt},j}(u_i)$  computed with the
    stored  $\delta\theta_{N,j}(u_i, z)$  by (41)
     $n_{\text{cnt}} \leftarrow n_{\text{cnt}} + 1$ 
  end for
end for
for  $j = n_{\text{red,joint}} + 1, \dots, n_{\text{red},\theta}$  do
   $Z_z(:, n_{\text{cnt}}) \leftarrow \delta z_{\text{plc},j}$  computed with the stored
   $\delta\theta_{N,j}(u_1, z), \dots, \delta\theta_{N,j}(u_{l_m}, z)$  by (42)
   $n_{\text{cnt}} \leftarrow n_{\text{cnt}} + 1$ 
end for
return  $Z_z$ 

```

It can be proved that the last columns of $Z_z(z_k)$ that represent the adjustment of placements $\delta z_{\text{plc},j}$ (42) are orthogonal to $\nabla_z e_{\text{pose}}(u, z_k)$:

Lemma 2. $\nabla_z e_{\text{pose}}^T \delta z_{\text{plc},j} = \mathbf{0}$.

Proof. See Appendix A.

Unfortunately, $\delta z_{\text{wpt},j}(u, z_k)$ from (41) is closely but not strictly orthogonal to $\nabla_z e_{\text{pose}}(z_k)$, because $0 \leq \mathbf{g}^T(u_i) \mathbf{g}(u_j) \leq \mathbf{g}^T(u_j) \mathbf{g}(u_j)$. Nevertheless, $\delta z_{\text{wpt},j}(u, z_k)$ can be slightly shaped to make it lie completely in the null-space of $\nabla_z e_{\text{pose}}(z_k)$ by the following projection:

$$\begin{aligned} & \tilde{\delta z}_{\text{wpt},j}(u, z_k) \\ &= \left(I - \left(\nabla_z e_{\text{pose}}^T(z_k) \right)^+ \nabla_z e_{\text{pose}}^T(z_k) \right) \delta z_{\text{wpt},j}(u, z_k). \end{aligned} \quad (43)$$

As such, all the columns $\delta z_{\text{plc},j}(z_k)$ and $\tilde{\delta z}_{\text{wpt},j}(u_i, z_k)$ with $i = 1, \dots, l_m$ and $j = 1, \dots, n_{\text{red},\theta}$ constitute the null-space basis $Z_z(z_k)$ that satisfies (28).

It can be concluded that all the columns of the null-space basis $Z_z(z_k)$ are continuous w.r.t. z_k if the Jacobians in (39) are non-singular. The singularity is avoided by the penalty

(16). This conclusion of the continuity is drawn by examining the dependency on $\mathbf{z}_k$ of the equations as follows:

$$\left. \begin{aligned} \mathbf{z}_k &\xrightarrow{(39)} J_c \xrightarrow{(40)} N_j \xrightarrow{(41)} \delta\theta_{N,j} \xrightarrow{(42)} \delta\mathbf{z}_{\text{wpt},j} \xrightarrow{(43)} \delta\tilde{\mathbf{z}}_{\text{wpt},j} \\ \delta\theta_{N,j} &\xrightarrow{(42)} \delta\mathbf{z}_{\text{plc},j} \end{aligned} \right\} Z_z(\mathbf{z}_k).$$

With the continuity property, the reduced Hessian approximation $\bar{B}_k$ by BFGS possesses sufficient accuracy (Coleman and Sorensen, 1984).

Full-dimensional gradient. In order to solve (31a, b), the gradient of the objective function $\nabla_z \tilde{\mathcal{J}}_m \in \mathbb{R}^{n_z}$ should be provided. In Ye et al. (2022), a finite difference for $Z_z(\mathbf{z}_k)^T \nabla_z \tilde{\mathcal{J}}_m \in \mathbb{R}^{n_z}$ is the computation bottle-neck of the whole optimization method. The present paper achieves a higher computation efficiency with the full-dimensional gradient by exploiting the automatic differentiation (AD) (Andersson et al., 2012).

Before using the AD, we divide $\tilde{\mathcal{J}}_m$ into $\tilde{\mathcal{J}}_{\text{analy},m}$ and $\tilde{\mathcal{J}}_{\text{AD},m}$. The gradient of the former can simply be solved analytically, while that of the latter cannot:

$$\tilde{\mathcal{J}}_m = \tilde{\mathcal{J}}_{\text{analy},m} + \tilde{\mathcal{J}}_{\text{AD},m}, \quad (44)$$

$$\nabla_z \tilde{\mathcal{J}}_m = \nabla_z \tilde{\mathcal{J}}_{\text{analy},m} + \nabla_z \tilde{\mathcal{J}}_{\text{AD},m}. \quad (45)$$

The AD only applies to the latter term $\tilde{\mathcal{J}}_{\text{AD},m}$. An observation on (23), (21), and (17) reveals what of $\tilde{\mathcal{J}}_m$ should be incorporated in $\tilde{\mathcal{J}}_{\text{analy},m}$ and $\tilde{\mathcal{J}}_{\text{AD},m}$, respectively:

$$\begin{aligned} \tilde{\mathcal{J}}_{\text{analy},m} &= w_{\text{flc}} \mathcal{J}_{\text{flc},m}(\mathbf{z}_m) \\ &+ \frac{1}{2} \|C_{m-1,1} \tilde{\mathbf{z}}_{m-1} - C_{m,0} \mathbf{z}_m + \boldsymbol{\lambda}_{m-1}\|_P^2 \\ &+ \frac{1}{2} \|C_{m,1} \mathbf{z}_m - C_{m+1,0} \tilde{\mathbf{z}}_{m+1} + \boldsymbol{\lambda}_m\|_P^2 \\ \tilde{\mathcal{J}}_{\text{AD},m} &= \frac{1}{l_m} \sum_{i=1}^{l_m} \tilde{\mathcal{J}}_{\text{AD},m,i}(\boldsymbol{\theta}(u_i, \mathbf{z})) \\ &= \frac{1}{l_m} \sum_{i=1}^{l_m} (w_{\text{stf}} \mathcal{J}_{\text{stf},m}(u_i, \mathbf{z}) + w_{\text{sing}} \mathcal{J}_{\text{sing},m}(u_i, \mathbf{z})). \end{aligned} \quad (46) \quad (47)$$

The major reason that the gradient of (47) cannot simply be solved analytically are the Jacobians $J_r(\boldsymbol{\theta})$, $r = 1, \dots, n_{\text{rob}}$ inside $\mathcal{J}_{\text{stf},m}$ and $\mathcal{J}_{\text{sing},m}$. It is difficult and computationally inefficient to obtain an analytical solution to $dJ_r(\boldsymbol{\theta})/d\boldsymbol{\theta}$. However, by providing the symbolic expression of the Jacobians using $\boldsymbol{\theta}$ as the symbolic variable, the whole

$\tilde{\mathcal{J}}_{\text{AD},m,i}$ which is dependent on the Jacobians can be processed by the AD at each sampled point along the discretized path u_i , $i = 1, \dots, l_m$. The output of the AD is $\nabla_{\boldsymbol{\theta}} \tilde{\mathcal{J}}_{\text{AD},m,i}(\boldsymbol{\theta}(u_i, \mathbf{z}))$, the gradient of $\tilde{\mathcal{J}}_{\text{AD},m,i}(\boldsymbol{\theta}(u_i, \mathbf{z}))$ w.r.t. $\boldsymbol{\theta}$. Finally, the gradient of (47) w.r.t. $\mathbf{z}$ is as follows, considering the parametrization with B-splines:

$$\begin{aligned} \nabla_z \tilde{\mathcal{J}}_{\text{AD},m}(\boldsymbol{\theta}(u_i, \mathbf{z})) \\ = \frac{1}{l_m} \sum_{i=1}^{l_m} G^T(u_i) \nabla_{\boldsymbol{\theta}} \tilde{\mathcal{J}}_{\text{AD},m,i}(\boldsymbol{\theta}(u_i, \mathbf{z})). \end{aligned}$$

Local convergence. Up till now, all the essential components to solve (24a, b, c) and consequently (31a, b) have been obtained. To update $\bar{B}_k$ of (31a) with the BFGS algorithm, the necessary secant equations are provided in Byrd and Nocedal (1990) which are no longer discussed here. Under the assumption of sufficiently accurate approximation of $\bar{B}_k$ with the available algorithms, the focus of the current sub-section is on the convergence of the reduced Hessian method in the robotic application.

As Byrd and Nocedal (1990) prove convergence with the help of merit functions in the line search, others such as Kupfer (1996) utilize the Lipschitz continuity and the mean value theorem in the horizontal and vertical steps. The former is not further considered because the optimization exhibits good convergence behavior when $\alpha = 1$ is achieved for (25) in the late stage, without relying on the line search operation. The present paper follows the latter idea but extends (Kupfer, 1996) by the bounding constraints of (15). These bounds remain in the linear inequality constrained quadratic programming (QP) problem (31b).

Problem (31a, b) is solved with the standard active-set method (Nocedal and Wright, 1999) which finds the indices of the activated constraints. If a number of $n_a \geq 0$ among the total $2n_z$ inequality constraints (31b) are activated at indices $i_a \in \mathcal{I}_a$, the activated constraints can be selected by the activation matrices $A_1, A_2 \in \mathbb{R}^{n_z \times n_a}$ which contain all zero but a single one at the i_a -th entry in each row. The activated constraints are expressed as follows:

$$\begin{bmatrix} A_1^T & A_2^T \end{bmatrix} \begin{bmatrix} Z_z(\mathbf{z}_k) \\ -Z_z(\mathbf{z}_k) \end{bmatrix} \bar{\mathbf{d}} = \begin{bmatrix} A_1^T & A_2^T \end{bmatrix} \begin{bmatrix} \boldsymbol{\theta}_{\max} - \mathbf{z}_k - \mathbf{v} \\ -\boldsymbol{\theta}_{\min} + \mathbf{z}_k + \mathbf{v} \end{bmatrix}$$

or by defining $A = A_1 - A_2$:

$$A^T Z_z(\mathbf{z}_k) \bar{\mathbf{d}} = A_1(\boldsymbol{\theta}_{\max} - \mathbf{z}_k - \mathbf{v}) - A_2(\boldsymbol{\theta}_{\min} - \mathbf{z}_k - \mathbf{v}). \quad (48)$$

Then the solution of (31a, b) and (30) can be expressed in the explicit form when the active set $\mathcal{I}_a$ is found and no longer changes after sufficient iterations:

$$\begin{aligned} \mathbf{h}(\mathbf{z}_k, \bar{B}_k) &= Z_z(\mathbf{z}_k) \bar{\mathbf{d}} \\ &= Z_z(\mathbf{z}_k) [\mathbf{v}_{\text{qp}}(\mathbf{z}_k) + \mathbf{h}_{\text{qp}}(\mathbf{z}_k, \bar{B}_k)], \end{aligned} \quad (49)$$

whereby the QP-vertical step

$$\mathbf{v}_{\text{qp}}(\mathbf{z}_k) = (A^T Z_z(\mathbf{z}_k))^+ \cdot [A_1(\boldsymbol{\theta}_{\max} - \mathbf{z}_k - \mathbf{v}) - A_2(\boldsymbol{\theta}_{\min} - \mathbf{z}_k - \mathbf{v})] \quad (50)$$

is the first step of the active set algorithm that finds a feasible point with respect to the activated bounds by solving (48). The QP-horizontal step

$$\mathbf{h}_{\text{qp}}(\mathbf{z}_k, \bar{\mathbf{B}}_k) = -Z_{\text{qp}} \left(Z_{\text{qp}}^T \bar{\mathbf{B}}_k Z_{\text{qp}} \right)^{-1} Z_{\text{qp}}^T Z_z(\mathbf{z}_k)^T \nabla_z \tilde{\mathcal{J}}_m \quad (51)$$

finds the QP-descent in the direction without violating (48) with the help of the QP-null-space basis $Z_{\text{qp}} = Z_{\text{qp}}(\mathbf{z}_k) \in \mathbb{R}^{(n_z - n_e) \times (n_z - n_e - n_a)}$. Z_{qp} is created using the QR decomposition of $(A^T Z_z(\mathbf{z}_k))^T$ and taking the $(n_a + 1)$ -th to the last columns of the orthonormal matrix Q . Although the resulting Z_{qp} might not be continuous w.r.t. $\mathbf{z}_k$ due to the algorithms in the QR decomposition, it is usable because Z_{qp} does not influence the approximation of $\bar{\mathbf{B}}_k$ in BFGS. Similar to (28) and (29), Z_{qp} satisfies:

$$A^T Z_z(\mathbf{z}_k) Z_{\text{qp}} = \mathbf{0}, \quad (52)$$

$$Z_{\text{qp}} Z_{\text{qp}}^+ + (A^T Z_z(\mathbf{z}_k))^+ A^T Z_z(\mathbf{z}_k) = I. \quad (53)$$

To prove the convergence as in Kupfer (1996), we observe the optimum point $\hat{\mathbf{z}}$, where both the vertical and horizontal step vanish but $\nabla_z \tilde{\mathcal{J}}_m$ of (51) does not. The only possibility is that $Z_{\text{qp}}^T Z_z(\mathbf{z}_k)^T \nabla_z \tilde{\mathcal{J}}_m = \mathbf{0}$ and $\mathbf{v}_{\text{qp}}(\mathbf{z}_k) = \mathbf{0}$. Therefore, $\mathbf{h}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) = \mathbf{0}$, regardless of $\bar{\mathbf{B}}$. Furthermore, we formulate the derivative of (25) w.r.t. $\mathbf{z}$ at $\hat{\mathbf{z}}$:

$$\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) = \nabla \mathbf{v}(\hat{\mathbf{z}}) + \alpha \nabla \mathbf{h}(\hat{\mathbf{z}}, \bar{\mathbf{B}}). \quad (54)$$

The derivatives of the vertical and horizontal step are given as follows according to (37) and (49):

$$\nabla \mathbf{v}(\hat{\mathbf{z}}) = -\left(\nabla_z \mathbf{e}_{\text{pose}}^T(\hat{\mathbf{z}}) \right)^+ \nabla_z \mathbf{e}_{\text{pose}}^T(\hat{\mathbf{z}}), \quad (55)$$

$$\nabla \mathbf{h}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) = Z_z(\hat{\mathbf{z}}) (\nabla \mathbf{v}_{\text{qp}}(\hat{\mathbf{z}}) + \nabla \mathbf{h}_{\text{qp}}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)). \quad (56)$$

According to (50) and (51), we further derive:

$$\nabla \mathbf{v}_{\text{qp}}(\hat{\mathbf{z}}) = (A^T Z_z(\mathbf{z}_k))^+ A^T [-I - \nabla \mathbf{v}(\hat{\mathbf{z}})], \quad (57)$$

$$\nabla \mathbf{h}_{\text{qp}}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) = -Z_{\text{qp}} \left(Z_{\text{qp}}^T \bar{\mathbf{B}} Z_{\text{qp}} \right)^{-1} Z_{\text{qp}}^T Z_z(\hat{\mathbf{z}})^T \hat{\mathbf{B}}, \quad (58)$$

whereby $\hat{\mathbf{B}} \in \mathbb{R}^{n_z \times n_z}$ is the true Hessian at $\hat{\mathbf{z}}$. Now we simplify $\nabla \mathbf{d}$ by combining (29) and (53) with (54)–(58). After some lengthy calculations, we get:

$$\begin{aligned} \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) &= \alpha \underline{\mathbf{Z}} \underline{\mathbf{Z}}^+ + (1 - \alpha) Z_z(\hat{\mathbf{z}}) Z_z(\hat{\mathbf{z}})^+ \\ &\quad - \alpha \underline{\mathbf{Z}} \left(Z_{\text{qp}}^T \bar{\mathbf{B}} Z_{\text{qp}} \right)^{-1} \underline{\mathbf{Z}}^T \hat{\mathbf{B}} - I, \end{aligned} \quad (59)$$

whereby we denote $\underline{\mathbf{Z}} = Z_z(\hat{\mathbf{z}}) Z_{\text{qp}}(\hat{\mathbf{z}})$ and $\underline{\mathbf{Z}}^+ = Z_{\text{qp}}(\hat{\mathbf{z}})^+ Z_z(\hat{\mathbf{z}})^+$. The inverse of $\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})$ is crucial for the later proof of convergence. This is also the case in Kupfer (1996) Lemma 3.2, but it considers neither the step length α , nor the activated inequality constraints. Therefore, the following lemma is necessary:

Lemma 3. The matrix $\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})$ as given by (59) is invertible, and the inverse is given as follows:

$$\begin{aligned} \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})^{-1} &= \underline{\mathbf{Z}} \left(\underline{\mathbf{Z}}^T \hat{\mathbf{B}} \underline{\mathbf{Z}} \right)^{-1} \left(\underline{\mathbf{Z}}^T \hat{\mathbf{B}} - \frac{1}{\alpha} Z_{\text{qp}}^T \bar{\mathbf{B}} Z_{\text{qp}} \underline{\mathbf{Z}}^+ \right) - I. \end{aligned} \quad (60)$$

Proof. For $0 < \alpha \leq 1$ as the step length, both $\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})^{-1} \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) = I$ and $\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})^{-1} = I$ can be verified merely using the property of the pseudo-inverse $\underline{\mathbf{Z}}^+ \underline{\mathbf{Z}} = I$. This also indicates that $\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})$ is invertible. $\square$

We further introduce the iteration function of the decision variable and its derivative:

$$\boldsymbol{\zeta}(\mathbf{z}, \bar{\mathbf{B}}) = \mathbf{z} + \mathbf{d}(\mathbf{z}, \bar{\mathbf{B}}), \quad (61)$$

$$\nabla \boldsymbol{\zeta}(\mathbf{z}, \bar{\mathbf{B}}) = I + \nabla \mathbf{d}(\mathbf{z}, \bar{\mathbf{B}}). \quad (62)$$

It applies iteratively $\mathbf{z}_{k+1} = \boldsymbol{\zeta}(\mathbf{z}_k, \bar{\mathbf{B}}_k)$. Finally, $\hat{\mathbf{z}}$ is a fixed point with $\boldsymbol{\zeta}(\hat{\mathbf{z}}, \bar{\mathbf{B}}) = \hat{\mathbf{z}}$ regardless of $\bar{\mathbf{B}}$. The convergence can be proved if we first establish the relation between iterates $\mathbf{z}_{k-1}$, $\mathbf{z}_k$, $\mathbf{z}_{k+1}$ and $\hat{\mathbf{z}}$. For that, the following assumptions of Lipschitz continuity are made as in Kupfer (1996).

Assumption 1. (Lipschitz Continuity)

For all $\bar{\mathbf{B}}$ and all $\mathbf{z}$ sufficiently close to a local optimum $\hat{\mathbf{z}}$, there exists a constant $\mu > 1$:

$$\|\nabla \mathbf{d}(\mathbf{z}, \bar{\mathbf{B}}) - \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})\| \leq \mu \|\mathbf{z} - \hat{\mathbf{z}}\|, \quad (63)$$

$$\|\mathbf{d}(\mathbf{z}, \bar{\mathbf{B}}) - \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})\| \leq \mu \|\mathbf{z} - \hat{\mathbf{z}}\|, \quad (64)$$

$$\|\boldsymbol{\zeta}(\mathbf{z}, \bar{\mathbf{B}}) - \boldsymbol{\zeta}(\hat{\mathbf{z}}, \bar{\mathbf{B}})\| \leq \mu \|\mathbf{z} - \hat{\mathbf{z}}\|, \quad (65)$$

$$\|\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})\| \leq \mu, \quad (66)$$

$$\|\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}})^{-1}\| \leq \mu. \quad (67)$$

As a further preparation for the establishment of the relation between iterates, the so-called mean value theorem is proved here (named but not explained in Kupfer (1996)).

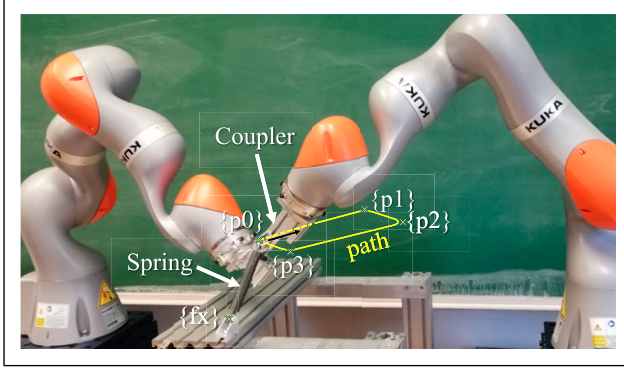


Figure 8. Two robots are physically coupled. The spring is connected to the fixture {fx} and the coupler. The connecting point between the spring and the coupler is denoted as TCP. The robots pull the spring-loaded TCP along the path in one loop. The starting point is {p0}. The TCP is currently at {p0}. The black arrow marks the starting direction of the path. {p1}, {p2}, and {p3} are the corners of the path, forming a parallelogram on the horizontal plane. The path is rounded at the corners.

Theorem 4. (The So-Called Mean Value Theorem)

For all $\mathbf{z} \in \mathbb{R}^{n_z}$, $\bar{\mathbf{B}} \in \mathbb{R}^{(n_z - n_c) \times (n_z - n_c)}$, there is:

$$\|\zeta(\mathbf{z}, \bar{\mathbf{B}}) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}) - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}})(\mathbf{z} - \hat{\mathbf{z}})\| \leq \frac{\mu}{2} \|\mathbf{z} - \hat{\mathbf{z}}\|^2. \quad (68)$$

Proof. See Appendix A.

Now we are ready to establish the relation between iterates $\mathbf{z}_{k-1}$, $\mathbf{z}_k$, $\mathbf{z}_{k+1}$ and $\hat{\mathbf{z}}$, similar but extended from Lemma 3.3 of Kupfer (1996) by the QP and line search.

Lemma 5. (Relation between $\mathbf{z}_{k-1}$, $\mathbf{z}_k$, $\mathbf{z}_{k+1}$, and $\hat{\mathbf{z}}$)

If the step length of the line search takes $\alpha = 1$ after sufficient number of iterations, there exists a constant $\beta > 0$ such that:

$$\|(\nabla \mathbf{v}(\hat{\mathbf{z}}) + \mathbf{Z}_z(\hat{\mathbf{z}}) \nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))(\mathbf{z}_{k+1} - \mathbf{z}_k)\| \leq \beta \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2. \quad (69)$$

Proof. See Appendix A.

By applying the intermediate inequality of Lemma 5, the following theorem of convergence is proved.

Theorem 6. (Two-Step Linear Convergence of the Reduced Hessian Method)

Table 1. Coordinates of points.

Point	x in m	y in m	z in m
{p0}	0	0	0
{p1}	0.1	0.18	0
{p2}	0.15	0.18	0
{p3}	0.05	0	0
{fx}	0	-0.08	-0.12

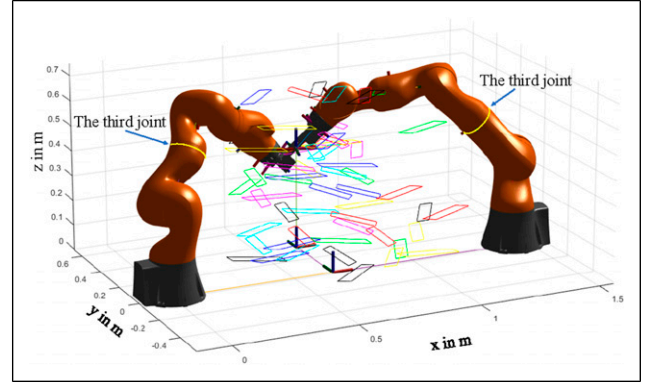


Figure 9. The placement of the virtual work-piece and the path with $n_{pw} = 4$ DOFs is sampled in the initial solutions and illustrated here. The robot placements and the redundant joints (marked as the third joints) are also sampled in a similar manner, but not illustrated.

After sufficient number of iterations, if the active set $\mathcal{I}_a$ of the QP-subproblem is found and no longer changes from iteration to iteration, and if the step length takes $\alpha = 1$, then for each given constant $0 < \rho < 1$, there exist a radius $\varepsilon_1 > 0$ and a reduced Hessian approximation accuracy $\xi > 0$ such that for $\|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\| \leq \varepsilon_1$, and for

$$\|\mathbf{Z}_z(\hat{\mathbf{z}})^T \hat{\mathbf{B}} \mathbf{Z}_z(\hat{\mathbf{z}}) - \bar{\mathbf{B}}_k\| < \xi, \quad (70)$$

the following applies:

$$\|\mathbf{z}_{k+1} - \hat{\mathbf{z}}\| \leq \rho \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|. \quad (71)$$

Remark 4. The null-space basis $\mathbf{Z}_z(\mathbf{z}_k)$ is continuous w.r.t. $\mathbf{z}_k$. According to Byrd and Nocedal (1990), under the condition of the continuity, sufficient approximation accuracy of $\bar{\mathbf{B}}_k$, as given by (70), can be reached using the BFGS algorithm.

Proof. See Appendix A.

If the sequence $\mathbf{z}_k$ with $k = 0, \dots, \infty$ converges to $\hat{\mathbf{z}}$, then the two-step superlinear convergence follows:

Corollary 7. (Two-Step Superlinear Convergence) After sufficient number of iterations, if the active set $\mathcal{I}_a$ of the QP-subproblem is found and no longer changes from iteration to iteration, if the step length takes $\alpha = 1$, and if the reduced Hessian approximation has

Table 2. Weights both for the new reduced Hessian method and for active set method of Matlab fmincon.

Max. iterations	w_{stf}	w_{sing}	w_{flc}
40	16.67	0.002	0.01

Table 3. Weights for the multi-stage successive constraint refinement.

Max. iterations	W_{stf}	W_{sing}	W_{flc}	W_{pose}
30	0	0.002	0	4.0
4	3.33	0.002	0.01	1.0
40	16.67	0.002	0.01	4.0
140	0	0.002	0	4.0

sufficient accuracy as ξ in (70) approaches zero, then the following applies:

$$\lim_{k \rightarrow \infty} \frac{\|z_{k+1} - \hat{z}\|}{\|z_k - \hat{z}\|} = 0. \quad (72)$$

Proof. See Appendix A.

Results

To illustrate the effectiveness of the trajectory planning, the following test cases are experimented with two physically coupled KUKA LBR iiwa 14 R820 robots:

- (i) pulling a spring
- (ii) multi-segment milling

Specifically, test case (i) verifies the core of the developed reduced Hessian method while utilizing the parallelization with multiple initial solutions. Test case (ii) utilizes the reduced Hessian method and further validates the ADMM inter-segment coordinated descend, as well as the stiffness model, so as to demonstrate the usability of the whole method set in reality. All the computations are conducted on a machine with the processor AMD Ryzen 7 PRO 6850U at 2.70 GHz.

Pulling a spring

The test case of pulling a spring imitates manufacturing processes with slowly varying process force such as sheet metal bending. This test case is also described in Ye et al. (2022) where the result with enhanced stiffness and reduced joint load after the optimization is shown. We use the same test case here to examine the robustness of the proposed reduced Hessian method with convergence guarantee in a statistical manner, as well as to show the improvement of efficiency compared to Ye et al. (2022).

Settings of the test case. Two robots should pull the spring along a given TCP path with the optimal stiffness against the force of the spring. The placement of the spring and the path w.r.t. the robots as well as the joint trajectories of robots are free for optimization, while the geometry of the path is pre-given. Figure 8 shows the settings. The given path is fixed to a work-piece frame. This work-piece frame

has $\{p0\}$ as the origin, line $\{p0\}-\{p3\}$ as the x -axis, and the vertical upward direction as the z -axis. The coordinates of the points related to the path are shown in Table 1. The spring has an original length 0.14 m and stiffness 600 N m^{-1} . It reaches the highest force of 141 N near $\{p2\}$ in Figure 8.

Generation of initial solutions. Multiple initial solutions are generated according to Algorithm 2. In the current test case, $n_{\text{seg}} = 1$. In the current and the next subsection, the joint trajectories are approximated by B-splines of order $\kappa = 3$, ensuring the continuity of angles, angular velocities and accelerations. Considering the length and

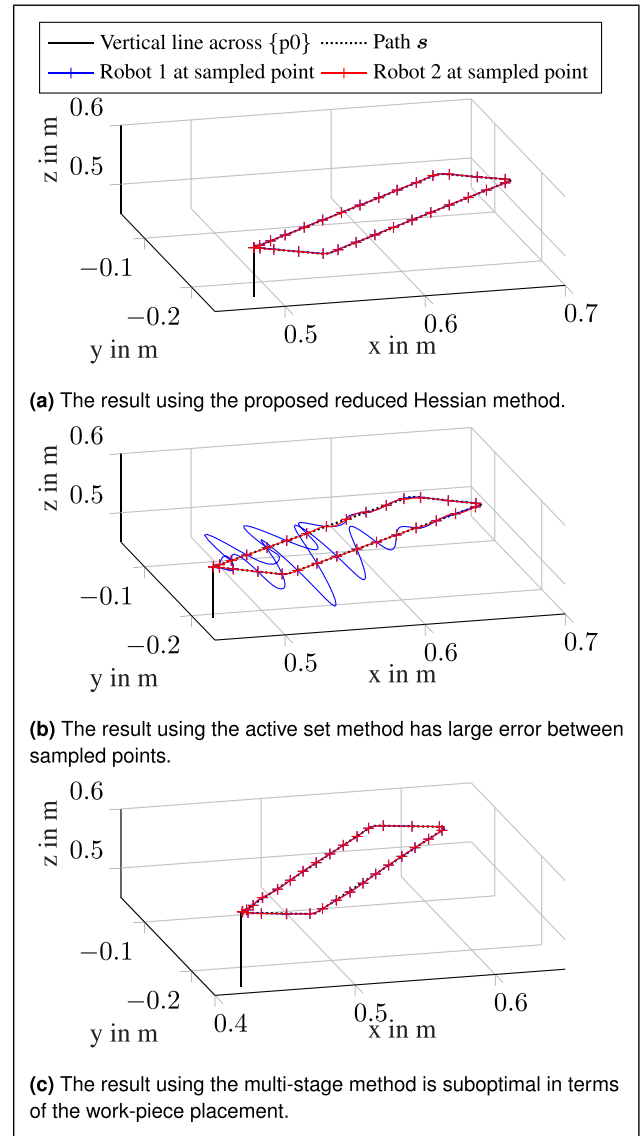


Figure 10. Visualization of representative results solved from the identical initial guess using the three methods. Only the reduced Hessian method shows both optimal work-piece placement and low path tracking errors.

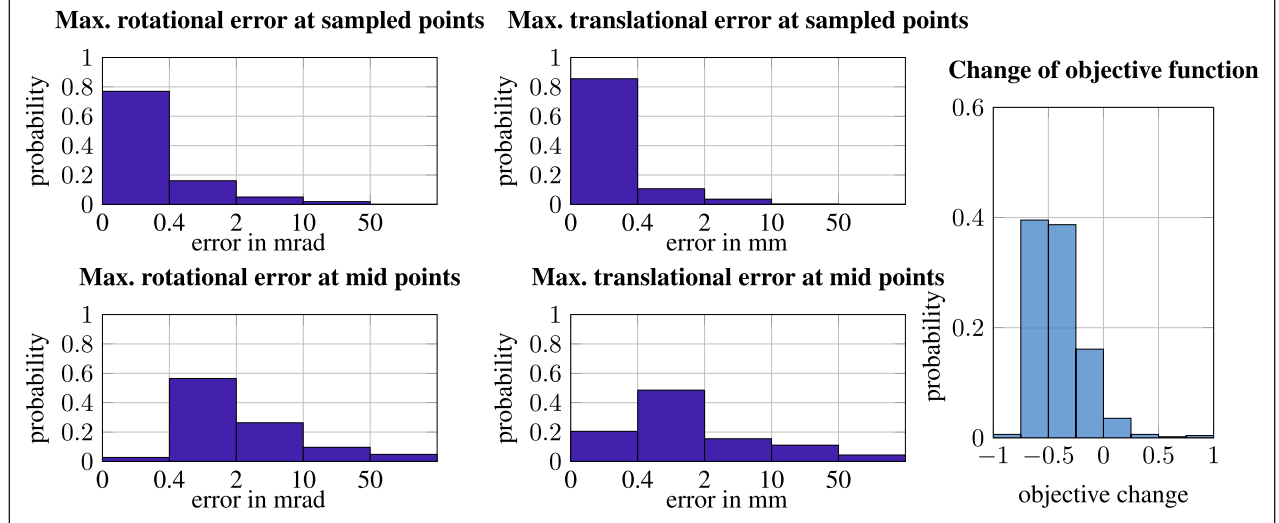


Figure 11. Distributions of path tracking errors and distribution of objective function change with the reduced Hessian method.

complexity of the TCP path, each joint trajectory has $l_1 = 30$ control points.

The configuration space of $\theta(u, z) = [q(u, z)^T, y^T]^T$ includes $q(u, z) \in \mathbb{R}^{14}$ of the two robots and $y \in \mathbb{R}^{n_{pr} + n_{pw}} = \mathbb{R}^5$. The dimension $n_{pr} = 1$ is the placement distance between the robots' bases. The dimensions $n_{pw} = 4$ include the 3D translational and the z-rotational placement of the path so that the path is kept on a horizontal plane. There are six dimensions of coupling constraints and six dimensions of path tracking constraints. Therefore, $14 + 5 - 6 - 6 = 7$ DOFs are available for the random sampling of initial solutions, including $y \in \mathbb{R}^5$ and two dimensions of redundant joints. The redundant joints are chosen to be the third joints, as shown in Figure 9, because they are at a position that does not exist in conventional 6-axis robots.

By sampling $n_{seed} = 5000$ and computing the function *aInvKin* in Algorithm 2, we get 478 initial solutions in which at least the starting point $\{p_0\}$ of the path is reachable by both robots. The positions of the path in the initial solutions, which correspond to $n_{pw} = 4$ elements of $y_0 \in \mathbb{R}^{n_{pr} + n_{pw}}$, are shown in Figure 9. In the current test case, function *invKin* in Algorithm 2 is not executed for subsequent way-points along the path to reduce the computation overhead, so in the initial solution, the robots have infeasible joint trajectories of constant values obtained from the function *aInvKin*, making the TCP stay static at $\{p_0\}$. This violation of the path tracking constraint (2) is only dealt with afterward in the optimization process.

Comparison of methods. Three methods are tested with the 478 initial solutions, namely the proposed reduced Hessian

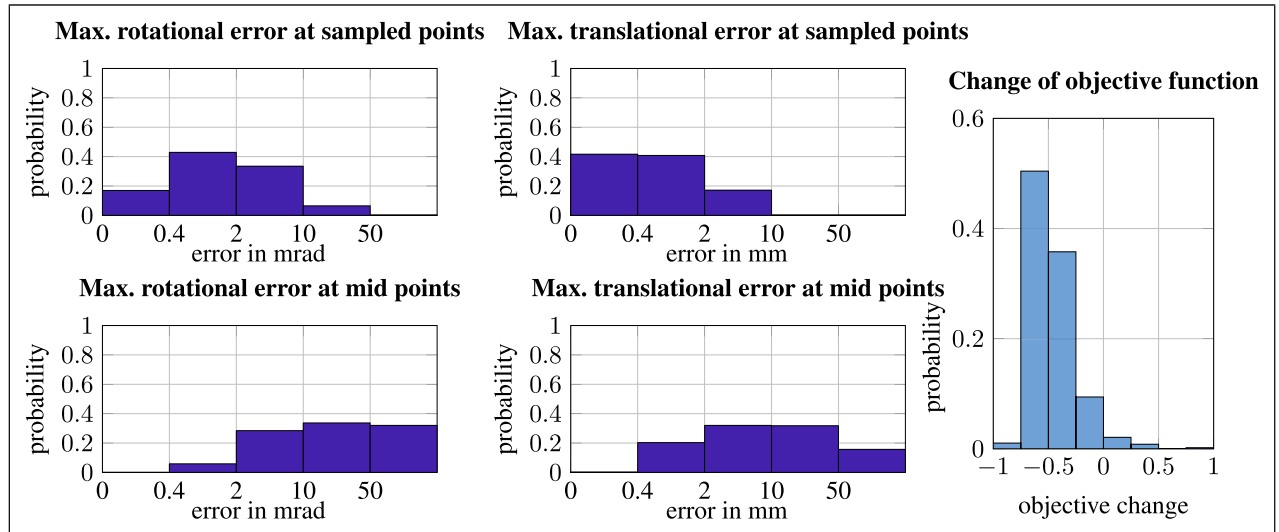


Figure 12. Distributions with the active set method from Matlab *fmincon*. The translational errors at middle points between the sampled points are large, indicating fluctuations as shown in Figure 10(b). These errors lead to a low success rate despite the significant reduction of the objective.

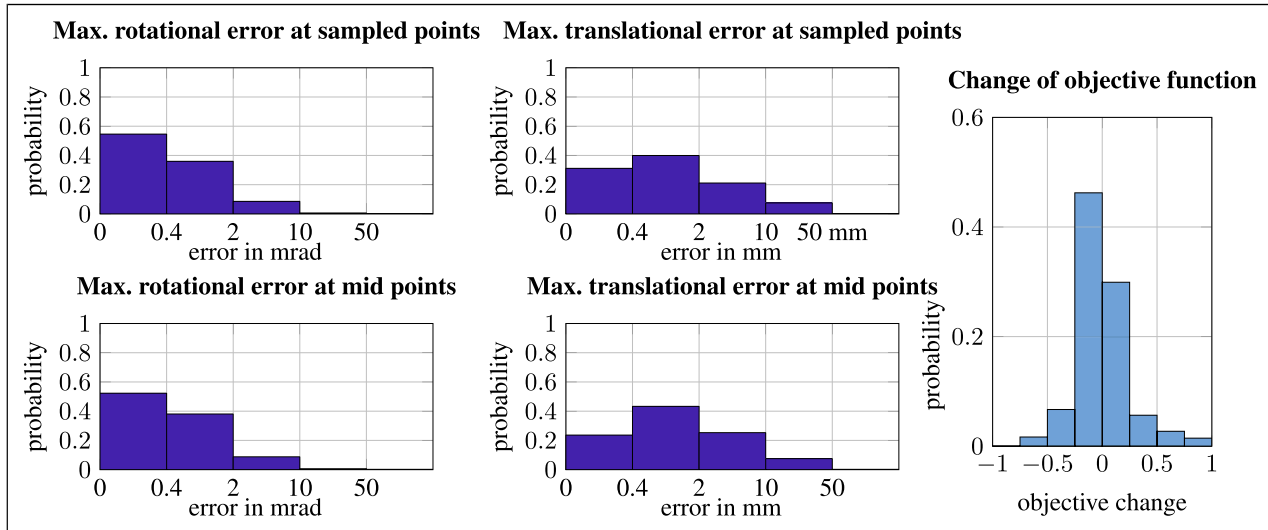


Figure 13. Distributions with the multi-stage successive constraint refinement method (Kabir et al., 2021). The errors are mostly kept small, similar to the ones of the reduced Hessian method. However, there is the difficulty in reducing the value of the objective function, which is also indicated in the difference of the work-piece placement in Figure 10(c).

method, the active set method in the Matlab built-in function *fmincon* (Nocedal and Wright, 1999), and the multi-stage successive constraint refinement (Kabir et al., 2021). The new reduced Hessian method is compared to the old one in Ye et al. (2022). The weights of (21) are given in Tables 2 and 3 for the three methods. In Table 3, the additional weight w_{pose} is the penalty weight for the pose error (14), since it is not handled as a constraint of the optimization problem.

From the 478 initial solutions, each method returns 478 results of the optimized placements and joint trajectories. From each method, one representative result is visualized in Figure 10, where the joint trajectories of both robots are transformed to the Cartesian TCP trajectories using the forward kinematics. These are compared against the path. The result of the reduced Hessian method has negligible errors at sampled points and small errors between sampled points. The result of the Matlab active set method has less error at, but larger error between sampled points. The result of the multi-stage successive constraint refinement method has small to medium errors both at and between sampled points. Moreover, in the multi-stage method, the placement of the work-piece is not optimized toward the position and orientation as in the other two methods, rendering the stiffness of robots suboptimal. The statistical results in Figures 11–13 confirm that the examples of Figure 10 are not rare individual cases.

Furthermore, the change of the objective function values between the initial and final solutions is captured in the histograms at the right of Figures 11–13. In all these evaluations of the objective function change, the weights in Table 2 apply. The medians of the change of the objective function values are given in Table 4. The objective function

value does not decrease significantly in most solutions of the multi-stage successive constraint refinement method and can even increase, although the errors are not larger than the reduced Hessian method. On the contrary, the Matlab active set method reduces the objective function value significantly, but the large errors render a large portion thereof unusable. Only the reduced Hessian method achieves a balance between the errors and the reduction of the objective function value. The success rates of the methods are given in Table 4. The optimization is deemed successful, if the rotational errors at and between sampled points are always below 0.01 rad, the translational errors at and between sampled points are always below 10 mm, and if the objective function value is reduced.

The reduced Hessian method also exhibits a shorter computing time of 8.08 s on average, as shown in Table 4. This computing time is acceptable for practical applications of manufacturing processes, if the planning and the motion control are separated in two stages, one offline in advance and the other in real-time. The computing time of the multi-stage successive constraint refinement method is 7.2 times

Table 4. Performance of methods among 478 results.

Methods	Objective function, %	Success rate, %	Computing time in s
Active set	−51.14	33.26	9.46
Multi-stage	−1.76	53.97	71.3
New red. Hess.	−39.94	78.87	8.08
Old red. Hess. (Ye et al., 2022)	−36.82	n.a.	65.89

Table 5. Ablation studies among 478 results.

Ablations of differentiation	Parallel computation	Computing time in s on average
No ablation, with AD	Turned on	8.08
Replacing AD by full finite difference	Turned on	17.46
Replacing AD by reduced finite difference	Turned on	13.53
With AD	Turned off	33.45
Replacing AD by reduced finite difference	Turned off	50.26

longer than our method, rendering it unacceptable in the current scenario.

The computing time sees a significant reduction compared to the old reduced Hessian method from Ye et al. (2022). The influencing factors for this reduction can be the differentiation methods, the parallel computation, and the improved convergence. The impacts of these factors are investigated in an ablation study as shown in Table 5. In the ablation of differentiation methods, AD is replaced by the full dimensional finite difference and the reduced finite difference. Compared to the AD, the full dimensional finite difference takes 116.1% longer time, and the reduced dimensional finite difference 67.5% longer. The influencing factor of the parallel computation contributes to an acceleration factor of four according to the comparison with the last two ablations in Table 5. The influence factor of the improved convergence is indicated by a comparison between the last row of Table 4 with the last row of Table 5 where the reduced finite difference and the lack of parallel computation are identical to the old testing conditions in Ye et al. (2022). But the computing time 50.26 s using the new reduced Hessian method is still 23.7% shorter than the result 65.89 s of the old one.

As a conclusion, the trade-offs in existing methods between the computational complexity and the whole-path optimality, as well as between the accurate constraint satisfaction and a high convergence speed are resolved by our method. We reach both a light computational complexity (embodied in the shorter computing time) and an improved whole-path optimality (reduction of objective function). Meanwhile, we also achieve an accurate constraint

Table 6. TCP position error in mm against control point numbers l_1 and B-spline orders κ .

$l_1 \backslash \kappa$	2	3	4	5	6	8	10
20	1.95	3.06	8.65	12.7	3.13	2.94	2.28
30	1.15	1.23	1.38	1.47	1.62	1.68	1.09
40	0.67	0.63	0.58	0.66	0.73	0.49	0.78
60	0.29	0.36	1.51	0.35	1.36	0.23	0.22
85	0.14	0.15	0.56	0.22	0.26	0.07	0.20

Table 7. Cost reduction in minus percentage against control point numbers l_1 and B-spline orders κ .

$l_1 \backslash \kappa$	2	3	4	5	6	8	10
20	47.5	42.9	49.5	48.0	43.4	43.7	0.02
30	43.2	43.1	43.7	42.4	43.3	39.1	44.7
40	41.5	42.4	43.5	41.9	41.7	40.6	40.9
60	40.9	40.3	40.6	38.4	37.1	38.6	38.7
85	35.6	37.2	29.5	25.1	26.6	42.7	38.9

satisfaction (high success rate in terms of limited constraint errors), as well as a high convergence speed.

Sensitivity of spline representation. To examine the robustness of the reduced Hessian method, a sensitivity analysis of spline representation is conducted analogue to Kabir et al. (2021). For the test case of pulling a spring, a range of number of control points l_1 from 20 to 85 and a range of B-spline order κ from two to ten are tested. For such a wide range, the regularizations for the vertical and horizontal steps in Remark 2 and Remark 3 contribute the greatest to the following results without large variations. The sensitivity tests are recorded in the color-mapped Tables 6–8. The background color **deep indigo** corresponds to a value with high performance, while the background color **golden-yellow** with a value in red corresponds to a critical one.

The position errors of the TCP from the given path are computed both at and between sampled points, among which the maximal value is given in Table 6. The position errors are kept below 2 mm for all cases except when the number of control points is at the lowest $l_1 = 20$ due to the lack of approximation capability of B-spline for the Cartesian path with insufficient control points. The cost reduction in Table 7 shows that the capability of the reduced Hessian method weakens gradually for high number of control points and high number of B-spline order. In one exceptional case with the lowest number of control points and the highest B-spline order, the method fails to descend. This can be explained due to the difficulty in the line search. As the step length does not approach $\alpha = 1$, no guarantee of Theorem 6 is provided. The computing time in Table 8 shows the general high efficiency

Table 8. Computing time in second against control point numbers l_1 and B-spline orders κ .

$l_1 \backslash \kappa$	2	3	4	5	6	8	10
20	7.97	3.37	10.9	23.0	6.59	7.76	181
30	7.07	5.71	9.08	9.19	16.5	10.2	67.6
40	16.9	15.3	25.1	20.6	25.3	17.8	32.7
60	33.5	44.6	141	165	284	32.8	42.8
85	84.1	140	650	165	652	223	108

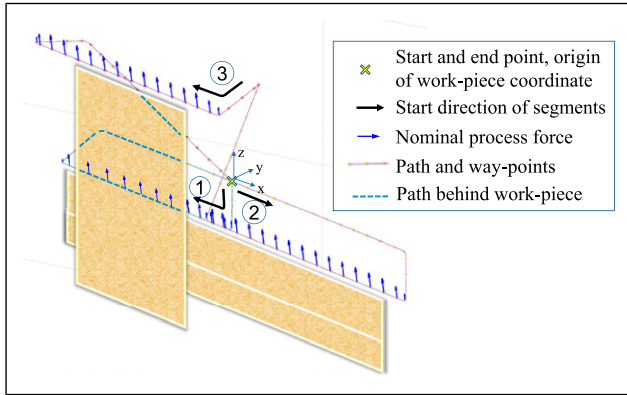


Figure 14. The path for the milling process.

of the method for number of control points lower or equal to 40 and B-spline order lower or equal to eight. The computations with less cost reduction generally require more computing time because of the difficulty in line search and consequently more iterations.

To further improve TCP tracking accuracy toward sub-millimeter region, one should leverage the online path tracking control (Ye et al., 2024) instead of refining the offline trajectory planning in the present paper. The error correction during the online tracking control works effectively as long as the position error is under 2 mm during the offline planning.

Multi-segment milling

To generate the milling trajectories, the proposed ADMM inter-segment coordinated descend is utilized and validated against the single-segment baseline. By executing the generated trajectories in the real-world milling, we demonstrate that the physical coupling and the trajectory optimization method set achieve a significant stiffness improvement.

Settings for ADMM validation. As shown in Figure 14, the three-dimensional path for the milling process consists of 11 straight lines and ten corners. On three of the straight lines, the milling head engages on the material, resulting in process forces. To approximate this path with sufficient accuracy, B-splines with 75 control points are used.

Table 9. Performance of the test instances after 30 iterations.

Number of segments	B-spline order	Computing time in s	Cost reduction, %	Max. error
1	3	138.2	−18.76	12.68 mm
1	4	173.1	−12.89	0.342 mm
3	3	204.1	−19.47	0.088 mm
3	4	209.1	−19.47	0.085 mm

To validate the ADMM inter-segment coordinated descend method, we split the path into three segments which are labeled with numbers in Figure 14. Inter-segment constraints are set to be (5) and (18) only. We compare the multi-segment planning results against the single-segment baseline. A variation of B-spline order between 3 and 4 is also made. For all these test instances, two robots are physically coupled and the TCP stiffness is optimized. No feasible initial solution guess is provided, since a feasible one is unlikely to be found manually or randomly in view of the complexity of the path. Instead, the optimization process starts with an arbitrary placement that renders the areas around the second and the ninth corner of the path unreachable within the joint limits.

ADMM computational results. The computational results of the aforementioned test instances are shown in Table 9. The single-segment planning with third-order B-splines results in a large error in path tracking and coupling, meaning it fails to find a feasible solution starting from the infeasible initial solution. The single-segment planning with fourth-order B-splines finds a feasible solution with inadequate cost reduction. The two results of the single-segment planning can be explained by the high complexity of the path, as well as the lack of an inter-segment slackness while pushing the solution toward a feasible optimum. Both of the two results of the 3-segment planning with the ADMM inter-segment coordinated descend are feasible and exhibit a larger cost reduction. There is no significant difference between the two results with different B-spline orders. However, comparing the computing time of Tables 8 and 9, the parallel computing of ADMM does not increase the computational efficiency due to the following reasons. To ensure the inter-segment continuity, the coordination between the three parallel threads requires additional computation. Despite the 30 iterations of the reduced Hessian method in all test instances, ADMM leads to a slower convergence rate of QP-subproblems and thus more iterations for QP and line search in the inner loop.

To reduce the computing time, we improve the convergence behavior using the fast ADMM with the Nesterov acceleration, as given in Algorithm 7 of Goldstein et al. (2014). However, our fast ADMM algorithm differs from Goldstein et al. (2014) in the use of the parallel computation, instead of the sequential one. This means, when calling the function *optimizeSingleSegment* and solving (23) for the m -th segment, the solution of, for example, the $(m - 1)$ -th segment $\tilde{z}_{m-1}$ is not the newest result but is from the last iteration. The Nesterov interpolation is only carried out if the augmented Lagrangians (23) of all segments decrease in the last iteration.

As a result, a faster cost reduction is achieved as shown in Table 10. We stop the iteration when the cost reduction is greater than the original ADMM method in Table 9. Instead

of 30 iterations there, the fast ADMM method is finished after 19 iterations. The computing time is halved despite the slight increase of the maximal error.

Further potential improvements are possible in the refinement of initial guesses, the adjustment of the frequency ratio between the updates of functions *optimizeSingleSegment* and *coordinateSegments* (see Algorithm 1), as well as tuning the coefficients of P in (23).

Further settings for the real milling. To demonstrate that the physical coupling and the trajectory planning method set achieve a significant stiffness improvement, we formulate and test the following two hypotheses:

Hypothesis 1. *Compared to a joint trajectory without stiffness optimization, the physically coupled robots moving along the optimized trajectory have a higher TCP stiffness.*

Hypothesis 2. *The physically coupled robots moving along their optimized joint trajectories have a higher TCP stiffness, compared to the TCP stiffness of a single robot moving along its own optimized trajectory.*

In the following real-world milling, the path for the milling process in Figure 14 still applies. All trajectories are generated by the 3-segment ADMM method with third-order B-splines.

To test Hypothesis 1, the placement and trajectories for the milling process with two physically coupled robots as in Figure 1 are optimized, once with and once without the non-zero stiffness term in the objective function, as given in (21) and (22). To test Hypothesis 2, the robot on the right-hand-side is detached from the coupler. The TCP stiffness of the remaining single-robot system with the coupler is optimized, as in Figure 15(a). The coupler attached to the single robot brings a disadvantageous lever arm from the TCP to the robot. To make a fair comparison, an additional stiffness optimized milling experiment with a shorter adapter is conducted, as shown in Figure 15(b).

In the milling experiments, joint space impedance control is used with the controller stiffness set to its maximum at $2000 \text{ N m rad}^{-1}$ for all joints. The resulting joint stiffness is analogue to the stiffness of two serially connected springs, one for the controller and the other for the mechanical joint. With the mechanical stiffness

identified in Busson et al. (2017), the resulting joint stiffness is shown in Table 11. The proportional-derivative-type impedance control mode is chosen against the proportional-integral-derivative-type position control for the quantitative experimental validation, because the stiffness values of the latter are unknown, non-configurable, and non-constant due to the integral term.

Remark 5. *We are aware that an accuracy-oriented set-point compensation for joint angles or TCP pose is possible in the practice (Ye et al., 2024), but this is not done in the milling experiments because the enhancement of absolute accuracy is not the purpose of the present paper and does not influence the result of stiffness evaluations. We focus on the stiffness enhancement which is the reduction of deviation under the process wrench.*

In the experiments, the process wrench is estimated with the joint torque sensors of the robots. After finding the difference of joint torques between a run with and a run without milling, this difference is transformed into the Cartesian space as the process wrench on TCP.

For two physically coupled robots, the resulting process wrench is the sum obtained from the two robots. The deviation under process wrench is estimated from the difference of link-side joint angles of robot $r = 1$ between a run with and a run without milling. This difference is then transformed into the Cartesian space to obtain the deviation on TCP. This estimation does not involve robots other than $r = 1$, because in this way, the small error in the placement of

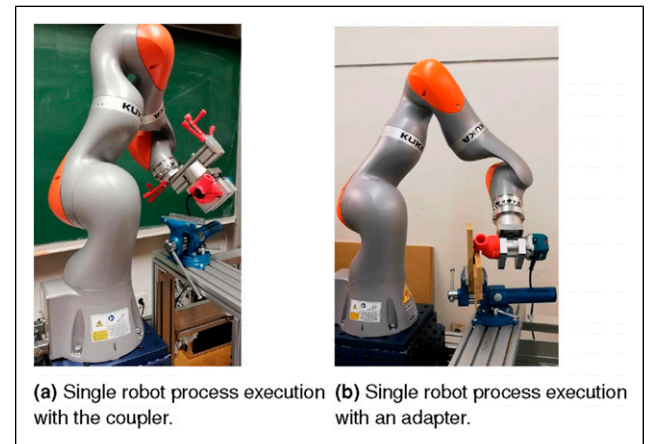


Figure 15. Settings of single-robot milling to compare against Figure 1 in testing Hypothesis 2.

Table 10. Performance of the Nesterov-accelerated fast ADMM for the three segments.

B-spline order	Iterations	Computing time in s	Cost reduction, %	Max. error
3	19	102.0	−19.50	0.117 mm
4	19	102.2	−19.51	0.087 mm

Table 11. Resulting joint stiffness in kN m rad^{-1} .

Joint	1	2	3	4	5	6	7
Value	1.91	1.90	1.72	1.82	1.60	1.31	1.41

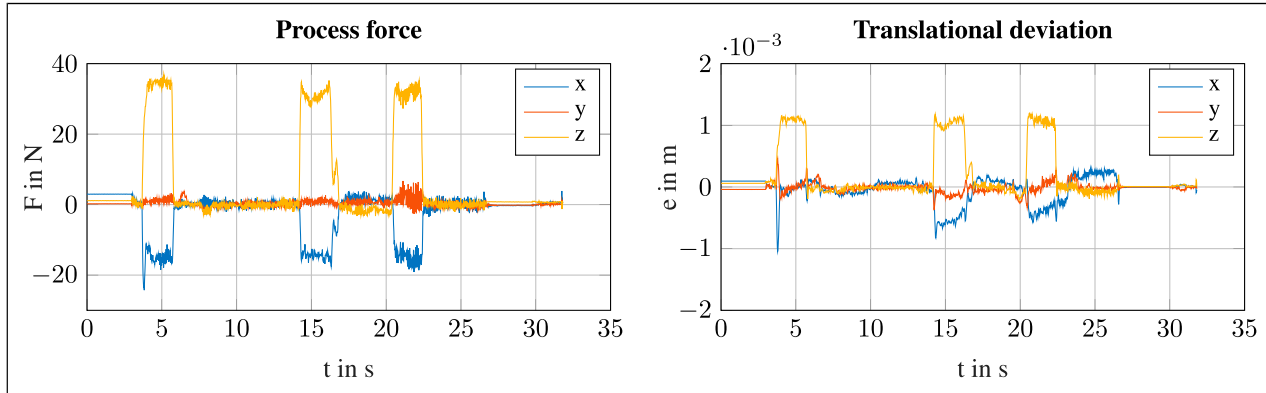


Figure 16. Experimental results with two robots in a physically coupled configuration after the optimization of TCP stiffness.

the second robot w.r.t. the first one has no influence on the estimation.

Milling results. Due to the variation of work-piece placement among different experiments, all the milling results are given in the work-piece coordinate system for a better comparability. Even in the same coordinate, the process wrench is not identical among the experiments, because the different TCP stiffness values lead to different deviations away from the material. As the TCP is engaged at different depth, the process wrench is also different. Despite this, the

TCP stiffness in different experiments, as a quotient between process wrench and deviation, is still comparable.

It is observed that the rotational components of the process wrench range from 0.5 N m to 2.0 N m for all the experiments, depending on the TCP stiffness and the depth of material engagement. Since this magnitude of the process torque is insignificant compared to that of the process force, we only compare the translational components in the following plots.

Hypothesis 1 is validated with a comparison between the milling experiment of Figure 16, Table 12 and that of

Table 12. Decision variables and stiffness values (in N mm^{-1}) from the milling experiment of Figure 16.

x_{wp}	y_{wp}	z_{wp}	ϕ_{wp}	$q_{13} (t = 0)$	$q_{23} (t = 0)$	K_{xx1}	K_{xx2}	K_{xx3}	K_{zz1}	K_{zz2}	K_{zz3}
0.642	-0.049	0.349 m	-0.491 rad	0.343 rad	0.283 rad	146	27.0	37.2	31.9	30.0	30.0

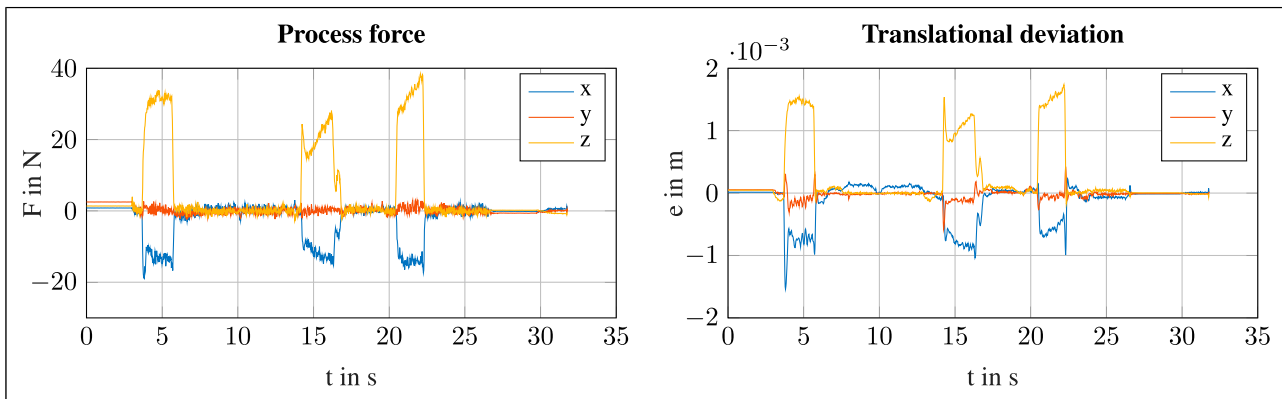


Figure 17. Experimental results with two robots in a physically coupled configuration without optimizing TCP stiffness.

Table 13. Decision variables and stiffness values (in N mm^{-1}) from the milling experiment of Figure 17.

x_{wp}	y_{wp}	z_{wp}	ϕ_{wp}	$q_{13} (t = 0)$	$q_{23} (t = 0)$	K_{xx1}	K_{xx2}	K_{xx3}	K_{zz1}	K_{zz2}	K_{zz3}
0.722	-0.155	0.350 m	-0.763 rad	0.242 rad	0.349 rad	16.8	14.8	30.8	21.5	19.4	22.2

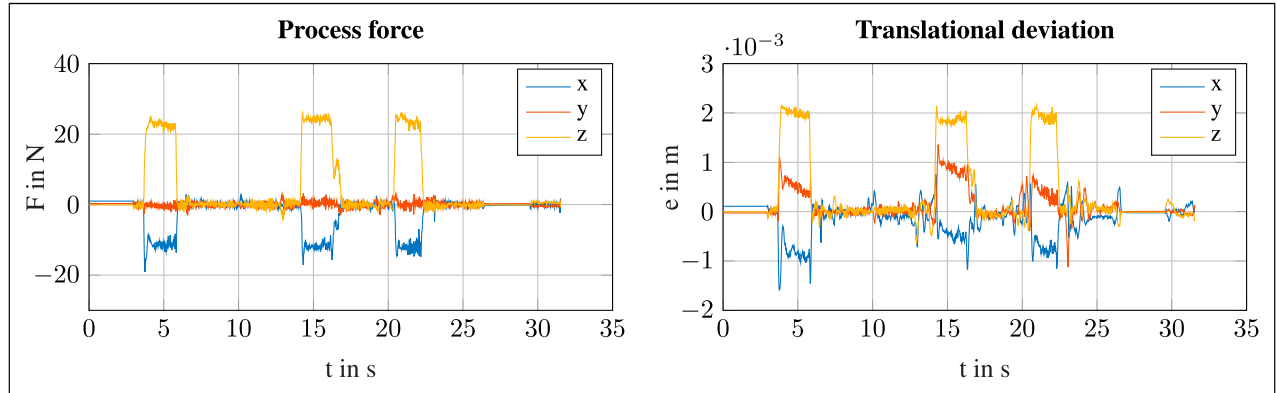


Figure 18. Experimental results with a single robot using the same coupler after the optimization of TCP stiffness.

Figure 17, Table 13. These two experiments both correspond to Figure 1, only with different work-piece placement and joint configurations. The different work-piece placements are given in the four parameters $y = [x_{wp}, y_{wp}, z_{wp}, \phi_{wp}]^T$ in Tables 12 and 13. The different joint configurations are characterized by the angles of the redundant joints, here designated as the third joint q_{13} of the first robot and q_{23} of the second robot. Only their values at $t = 0$ are given in the tables. As the robots move along the milling path with changing joint configurations, the TCP stiffness is non-constant. Therefore, average values of directional stiffness $K_{xx,m}$ (x -force component to x -deviation, in direction of feed) and $K_{zz,m}$ (z -force component to z -deviation, in direction of cutting depth) are taken in work-piece coordinate system for the three milling segments $m = 1, 2, 3$ separately.

The results show that all the stiffness values in the optimized case are higher than the ones without optimization by at least 20.8% in x -direction and at least 35.1% in z -direction. Therefore, Hypothesis 1 is valid in the current test case.

Hypothesis 2 is validated with a comparison between the milling experiment of Figure 16, Table 12 and that of Figure 18, Table 14. In both experiments, the coupler is used, as shown in Figures 1 and 15(a). Additionally, a comparison between the milling experiment of Figure 16, Table 12 and that of Figure 19, Table 15 is provided, corresponding to Figures 1 and 15(b), where the coupler is replaced by a shorter adapter. The results show that the optimized stiffness of two physically coupled robots compared to a single robot using the same coupler is higher by at least 9.8% in x -direction, at least 136% in z -direction. Apart from one exception in x -direction, the stiffness is more than

Table 14. Decision variables and stiffness values (in N mm^{-1}) with a single robot using the same coupler after the optimization of TCP stiffness.

x_{wp}	y_{wp}	z_{wp}	ϕ_{wp}	$q_{13} (t = 0)$	$q_{23} (t = 0)$	K_{xx1}	K_{xx2}	K_{xx3}	K_{zz1}	K_{zz2}	K_{zz3}
0.324	0.024	0.342 m	-1.106 rad	0.256 rad	n.a.	13.1	24.6	15.2	11.4	12.7	11.1

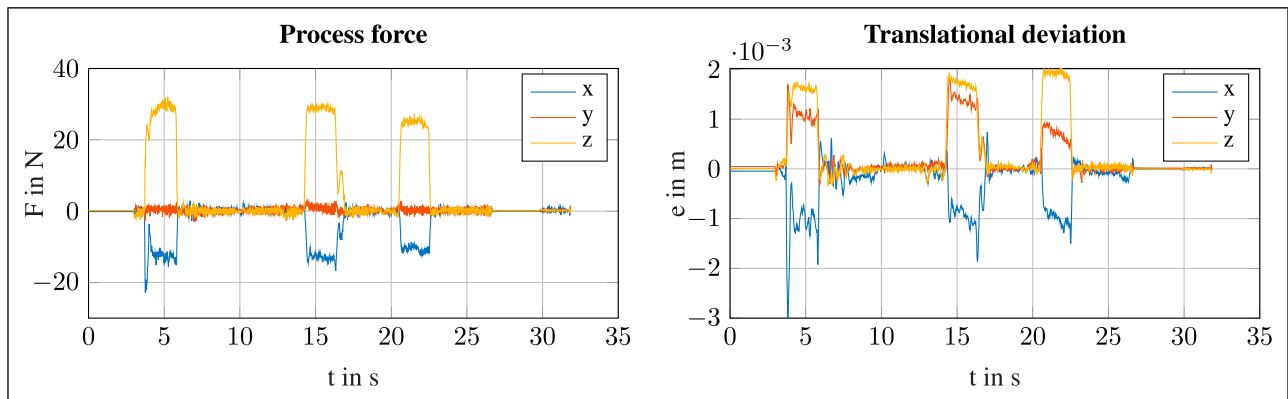


Figure 19. Results with a single robot using a shorter adapter after the optimization of TCP stiffness.

Table 15. Decision variables and stiffness values (in N mm^{-1}) with a single robot using a shorter adapter after the optimization of TCP stiffness.

x_{wp}	y_{wp}	z_{wp}	ϕ_{wp}	$q_{13} (t = 0)$	$q_{23} (t = 0)$	K_{xx1}	K_{xx2}	K_{xx3}	K_{zz1}	K_{zz2}	K_{zz3}
0.309	0.016	0.271 m	-1.327 rad	0.222 rad	n.a.	11.1	13.3	11.9	17.8	16.8	12.9

doubled. Compared to a single robot using the shorter adapter, the optimized stiffness of two physically coupled robots is still always higher. Therefore, Hypothesis 2 is valid in the current test case. Moreover, there exists a correlation between the y -component of translational deviation and the process force in x and z directions for a single robot, which is less significant for physically coupled robots.

The stiffness optimization in all above experiments is carried out assuming that the coupler is a rigid body, with $K_{c,r}^{-1} = 0, r = 1, 2$ in (8), because its stiffness values are difficult to determine. In the test instance of Table 12, the theoretical TCP stiffness values for two physically coupled robots in z -direction should be $K_{zz1} = 38.9 \text{ N mm}^{-1}$, $K_{zz2} = 47.5 \text{ N mm}^{-1}$, and $K_{zz3} = 36.9 \text{ N mm}^{-1}$.

Remark 6. *The real-world stiffness is lower than the theoretical calculation, indicating the influence of the unmodeled coupler elasticity on the resulting TCP stiffness. This elasticity can be affirmed by the experimental observation. On average among three milling segments in the experiment after the stiffness optimization, the process force induces an internal torque in the coupler of 28.1 N m, resulting in the rotational deformation of 2.07×10^{-3} rad. This rotational compliance of the coupler lowers the translational TCP stiffness. In case of the coupler used in Ye et al. (2022) with even lower stiffness, the TCP stiffness of two physically coupled robots can be lower than that of a single robot in Figure 15(b).*

Conclusion

Methodologically, the constrained trajectory planning using our reduced Hessian method resolves the conflict in existing methods between computational complexity and whole-path optimality, as well as between accurate constraint satisfaction and high convergence speed. Practically, in robot-driven manufacturing processes, the present paper gives a positive answer to the feasibility question on the concept of physically coupling multiple robots for capability enhancements. The concept is made feasible in practice, if the complete method set provided in the present paper is applied.

The proposed method set consists of an efficient reduced Hessian optimization method in its core for the constrained trajectory planning, as well as various supporting techniques to evaluate the collective capabilities, approximate smooth motion trajectories, consider motion limits of robots, deal with highly complex manufacturing paths, and to handle multiple local

optima in the robot configurations. The core reduced Hessian method is developed with the adaptation to robotic use cases where path and coupling constraints, B-spline trajectory approximation and motion limits are integrated. With these changes, the convergence guarantee in the literature does not apply anymore, but is newly given in the present paper, constituting a theoretical contribution. This advancement in theory is accompanied by an intensive empirical study of the proposed optimization method in comparison with the state-of-the-art, demonstrating the highest success rate of 78.87%, a large reduction of the objective function, low errors in coupling and path tracking, as well as reasonably low correlations of its performance w.r.t. variations in the trajectory parameterization. The practical problem of physical coupling is addressed in the real-world test case of robot-driven milling process. Despite the challenges in the complex path and the multiplicity of solutions in robotic configurations, the trajectory planning is made possible by the method set with an extension of ADMM parallel computing. Finally, it is shown that the capability of physically coupled robots is enhanced significantly with the proposed method set.

Acknowledgments

The authors thank Robert Wilbrandt for valuable comments and suggestions and Yuying Zhao for the assisting work in the research.

ORCID iDs

Xin Ye  <https://orcid.org/0000-0001-8067-2613>

Tobias Schürmann  <https://orcid.org/0000-0002-1967-7076>

Stefan Schwab  <https://orcid.org/0000-0002-2646-7755>

Author contributions

Xin Ye conceptualized the research, developed the methods, did the formal analysis and proofs, and wrote the original draft. Max Grobber and Tobias Schürmann provided critical reviews and corrections to the manuscript. Stefan Schwab provided critical reviews and corrections and revised the manuscript prior to the publication. Sören Hohmann conceptualized the research, revised the formal analysis and proofs, and provided critical reviews and corrections to the manuscript.

Funding

The authors disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was funded by the Federal Ministry of Education and Research (BMBF) and the Baden-Württemberg Ministry of Science as part of the Excellence Strategy of the German Federal and State Governments. This work was conducted within the

KIT Future Fields project Wertstromkinematik (www.wertstromkinematik.de) of the DFG Excellence Initiative.

Declaration of conflicting interests

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

References

- Abele E, Weigold M and Kulok M (2006) Increasing the accuracy of a milling industrial robot. *Production Engineering* 13(2): 221–224.
- Abele E, Weigold M and Rothenbücher S (2007) Modeling and identification of an industrial robot for machining applications. *CIRP Annals* 56(1): 387–390.
- Aginaga J, Zabalza I, Altuzarra O, et al. (2012) Improving static stiffness of the 6-RUS parallel manipulator using inverse singularities. *Robotics and Computer-Integrated Manufacturing* 28(4): 458–471.
- Andersson J, Åkesson J, and Diehl M. CasADi: A Symbolic Package for Automatic Differentiation and Optimal Control. In: *Recent Advances in Algorithmic Differentiation*. Edited by Forth S, Hovland P, Phipps E, Utke J, and Walther A, 297–307 Berlin, Heidelberg: Springer, 2012.
- Bachmann T, Nottensteiner K and Roa MA (2021) Automated planning of workcell layouts considering task sequences. In: 2021 IEEE International Conference on Robotics and Automation (ICRA), Xi'an, China, 30 May 2021–05 June 2021, pp. 12662–12668. IEEE.
- Bonalli R, Bylard A, Cauligi A, et al. (2019) Trajectory optimization on manifolds: a theoretically-guaranteed embedded sequential convex programming approach. ArXiv Preprint arXiv:1905.07654.
- Boumal N (2023) *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press.
- Boumal N, Mishra B, Absil PA, et al. (2014) Manopt, a Matlab toolbox for optimization on manifolds. *Journal of Machine Learning Research* 15(1): 1455–1459.
- Boyd S, Parikh N, Chu E, et al. (2011) Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends® in Machine Learning* 3(1): 1–122.
- Busson D, Bearee R and Olabi A (2017) Task-oriented rigidity optimization for 7 DOF redundant manipulators. *IFAC-PapersOnLine* 50(1): 14588–14593.
- Byrd RH and Nocedal J (1990) An analysis of reduced hessian methods for constrained optimization. *Mathematical Programming* 49(1): 285–323.
- Chiu SL (1988) Task compatibility of manipulator postures. *The International Journal of Robotics Research* 7(5): 13–21.
- Coleman TF and Sorensen DC (1984) A note on the computation of an orthonormal basis for the null space of a matrix. *Mathematical Programming* 29(2): 234–242.
- Conejo AJ, Castillo E, Minguez R, et al. (2006) *Decomposition Techniques in Mathematical Programming: Engineering and Science Applications*. Springer Science & Business Media.
- Curtin W and Scher H (1990) Mechanics modeling using a spring network. *Journal of Materials Research* 5(3): 554–562.
- De Maeyer J, Moyaers B and Demeester E (2017) Cartesian path planning for arc welding robots: evaluation of the descartes algorithm. In: 2017 22nd IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), Limassol, Cyprus, 12–15 September 2017, pp. 1–8. IEEE.
- Diaz Posada JR, Schneider U, Sridhar A, et al. (2017) Automatic motion generation for robotic milling optimizing stiffness with sample-based planning. *Machines* 5(1): 3.
- Erhart S and Hirche S (2016) Model and analysis of the interaction dynamics in cooperative manipulation tasks. *IEEE Transactions on Robotics* 32(3): 672–683.
- Gabay D (1982) Minimizing a differentiable function over a differential manifold. *Journal of Optimization Theory and Applications* 37: 177–219.
- Goldstein T, O'Donoghue B, Setzer S, et al. (2014) Fast alternating direction optimization methods. *SIAM Journal on Imaging Sciences* 7(3): 1588–1623.
- Han L (2001) A kinematics-based probabilistic roadmap method for closed chain systems. *Algorithmic and Computational Robotics: New Directions*: 233–246.
- Hoschek J and Lasser D (1993) *Fundamentals of Computer Aided Geometric Design*. AK Peters, Ltd.
- Jaillet L and Porta JM (2012) Path planning under kinematic constraints by rapidly exploring manifolds. *IEEE Transactions on Robotics* 29(1): 105–117.
- Jaquier N and Rozo L (2020) High-dimensional bayesian optimization via nested riemannian manifolds. *Advances in Neural Information Processing Systems* 33: 20939–20951.
- Jaquier N, Rozo L, Calinon S, et al. (2020) Bayesian optimization meets riemannian manifolds in robot learning. *Proceedings of the Conference on Robot Learning*. Proceedings of Machine Learning Research (PMLR), 100, 233–246.
- Kabir AM, Langsfeld JD, Kaipa KN, et al. (2018) Identifying optimal trajectory parameters in robotic finishing operations using minimum number of physical experiments. *Integrated Computer-Aided Engineering* 25(2): 111–135.
- Kabir AM, Kanyuck A, Malhan RK, et al. (2019) Generation of synchronized configuration space trajectories of multi-robot systems. In: 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019, pp. 8683–8690. IEEE.
- Kabir AM, Thakar S, Malhan RK, et al. (2021) Generation of synchronized configuration space trajectories with workspace path constraints for an ensemble of robots. *The International Journal of Robotics Research* 40(2-3): 651–678.
- Kalakrishnan M, Chitta S, Theodorou E, et al. (2011) STOMP: stochastic trajectory optimization for motion planning. In: 2011 IEEE international conference on robotics and automation, Shanghai, China, 09–13 May 2011, pp. 4569–4574. IEEE.
- Kavraki LE, Svestka P, Latombe JC, et al. (1996) Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Transactions on Robotics and Automation* 12(4): 566–580.

- Kicki P, Liu P, Tateo D, Bou-Ammar H, Walas K, Skrzypczyński P and Peters J (2023) Fast kinodynamic planning on the constraint manifold with deep neural networks. *IEEE Transactions on Robotics* 40: 277–297.
- Kingston Z, Moll M and Kavraki LE (2019) Exploring implicit spaces for constrained sampling-based planning. *The International Journal of Robotics Research* 38(10-11): 1151–1178.
- Kline M (1998) Taylor's theorem and infinite series. *Calculus: An Intuitive and Physical Approach*. Mineola, New York: Dover Publications, Inc., 637–642.
- Kunoth A (2018) Adaptive multiscale methods for the numerical treatment of systems of pdes. In: *Splines and PDEs: From Approximation Theory to Numerical Linear Algebra*. Edited by Kunoth A, Lyche T, Sangalli G, Serra-Capizzano S, Manni C, and Speleers H, 77–159 Springer.
- Kupfer FS (1996) An infinite-dimensional convergence theory for reduced sqp methods in hilbert space. *SIAM Journal on Optimization* 6(1): 126–163.
- LaValle SM and Kuffner JJJ (2001) Randomized kinodynamic planning. *The International Journal of Robotics Research* 20(5): 378–400.
- Lin Y, Zhao H and Ding H (2017) Posture optimization methodology of 6r industrial robots for machining using performance evaluation indexes. *Robotics and Computer-Integrated Manufacturing* 48: 59–72.
- Lu L, Zhang J, Tian X, et al. (2021) Tool path optimization for robotic surface machining by using sampling-based motion planning algorithms. *Journal of Manufacturing Science and Engineering* 143(1): 011002.
- Lyche T, Manni C and Speleers H. (2018) *Foundations of spline theory: B-splines, spline approximation, and hierarchical refinement*. In: *Splines and PDEs: From Approximation Theory to Numerical Linear Algebra*. Edited by Kunoth A, Lyche T, Sangalli G, Serra-Capizzano S, Manni C and Speleers H. (pp. 1–76), Springer, 2018.
- Malhan RK, Kabir AM, Shah B, et al. (2019) Identifying feasible workpiece placement with respect to redundant manipulator for complex manufacturing tasks. In: 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019, pp. 5585–5591. IEEE.
- Martin BJ and Bobrow JE (1999) Minimum-effort motions for open-chain manipulators with task-dependent end-effector constraints. *The International Journal of Robotics Research* 18(2): 213–224.
- Menasri R, Nakib A, Oulhadj H, et al. (2013) Path planning for redundant manipulators using metaheuristic for bilevel optimization and maximum of manipulability. In: 2013 IEEE international conference on Robotics and Biomimetics (ROBIO), Shenzhen, China, 12–14 December 2013, pp. 145–150. IEEE.
- Mühlbeier E, Gönnheimer P, Hausmann L, et al. (2021) Value stream kinematics. In: *Production at the leading edge of technology: Proceedings of the 10th congress of the german academic association for production technology (WGP)*, Dresden, 23–24 September 2020, pp. 409–418.
- Mutti S, Nicola G, Beschi M, et al. (2021) Towards optimal task positioning in multi-robot cells, using nested meta-heuristic swarm algorithms. *Robotics and Computer-Integrated Manufacturing* 71: 102131.
- Neumaier A (1998) Solving ill-conditioned and singular linear systems: a tutorial on regularization. *SIAM Review* 40(3): 636–666.
- Nicola G, Pedrocchi N, Mutti S, et al. (2018) Optimal task positioning in multi-robot cells, using nested meta-heuristic swarm algorithms. *Procedia CIRP* 72: 386–391.
- Nocedal J and Wright SJ (1999) *Numerical Optimization*. Springer.
- Pan Z, Wang D, Deng H, et al. (2019) A virtual spring method for the multi-robot path planning and formation control. *International Journal of Control, Automation and Systems* 17: 1272–1282.
- Pennec X, Fillard P and Ayache N (2006) A riemannian framework for tensor computing. *International Journal of Computer Vision* 66: 41–66.
- Perzylo A, Rickert M, Kahl B, et al. (2019) Smerobotics: smart robots for flexible manufacturing. *IEEE Robotics and Automation Magazine* 26(1): 78–90.
- Piegl L and Tiller W (2012) *The NURBS Book*. Springer Science & Business Media.
- Qureshi AH, Dong J, Baig A, et al. (2021) Constrained motion planning networks x. *IEEE Transactions on Robotics* 38(2): 868–886.
- Ranjbaran F, Angeles J and Kecskeméthy A (1996) On the kinematic conditioning of robotic manipulators. *Proceedings of IEEE International Conference on Robotics and Automation* 4: 3167–3172.
- Schmid C and Biegler LT (1994) Quadratic programming methods for reduced hessian sqp. *Computers & Chemical Engineering* 18(9): 817–832.
- Schulman J, Duan Y, Ho J, et al. (2014) Motion planning with sequential convex optimization and convex collision checking. *The International Journal of Robotics Research* 33(9): 1251–1270.
- Shimizu M, Kakuya H, Yoon WK, et al. (2008) Analytical inverse kinematic computation for 7-dof redundant manipulators with joint limits and its application to redundancy resolution. *IEEE Transactions on Robotics* 24(5): 1131–1142.
- Sintov A, Borum A and Bretl T (2018) Motion planning of fully actuated closed kinematic chains with revolute joints: a comparative analysis. *IEEE Robotics and Automation Letters* 3(4): 2886–2893.
- Takacs S and Takacs T (2016) Approximation error estimates and inverse inequalities for b-splines of maximum smoothness. *Mathematical Models and Methods in Applied Sciences* 26(07): 1411–1445.
- Tang Y, Shen W, Mamaev I, et al. (2023) Towards flexible manufacturing: motion generation concept for coupled multi-robot systems. In: 2023 IEEE 19th international Conference on Automation Science and Engineering (CASE), Auckland, New Zealand, 26–30 August 2023, pp. 1–7. IEEE.
- Ugray Z, Lasdon L, Plummer J, et al. (2007) Scatter search and local nlp solvers: a multistart framework for global optimization. *INFORMS Journal on Computing* 19(3): 328–340.
- Wang Y, Wang S and Wu L (2017) Distributed optimization approaches for emerging power systems operation: a review. *Electric Power Systems Research* 144: 127–135.

- Weingartshofer T, Bischof B, Meiringer M, et al. (2023) Optimization-based path planning framework for industrial manufacturing processes with complex continuous paths. *Robotics and Computer-Integrated Manufacturing* 82: 102516.
- Yakey JH, LaValle SM and Kavraki LE (2001) Randomized path planning for linkages with closed kinematic chains. *IEEE Transactions on Robotics and Automation* 17(6): 951–958.
- Ye X, Majer N, Schwab S and Hohmann S (2025) Real-world accuracy improvement for cooperative manipulation using model predictive control. *IFAC-PapersOnLine* 59(17): 209–214.
- Ye X, Schwartz M and Hohmann S (2022) Stiffness optimized multi-robot behavior planning using reduced hessian method. *IFAC-PapersOnLine* 55(38): 55–60.
- Ye X, Handwerker K and Hohmann S (2024) Adaptive model predictive control for differential-algebraic systems towards a higher path accuracy for physically coupled robots. In: 2024 IEEE/RSJ international conference on Intelligent Robots and Systems (IROS), Abu Dhabi, United Arab Emirates, 14–18 October 2024, pp. 8497–8503. IEEE.
- Zha F, Liu Y, Guo W, et al. (2018) Learning the metric of task constraint manifolds for constrained motion planning. *Electronics* 7(12): 395.
- Zucker M, Ratliff N, Dragan AD, et al. (2013) Chomp: covariant hamiltonian optimization for motion planning. *The International Journal of Robotics Research* 32(9-10): 1164–1193.

Appendix

Appendix A: Proofs

Lemma 1. $\nabla_{\theta} \mathbf{e}_{\text{pose}}^T \delta \theta_{N,j}(u, \mathbf{z}_k) = \mathbf{0}$.

Proof. Denote $\nabla_{\theta} \mathbf{e}_{\text{pose}}^T(u, \mathbf{z}_k) = J_e$, therefore,

$$\begin{aligned} & \nabla_{\theta} \mathbf{e}_{\text{pose}}^T \delta \theta_{N,j}(u, \mathbf{z}_k) \\ &= J_e (I - J_e^+ J_e) \delta \theta_{\text{red},j} \\ &= J_e (I - [J_e^T (J_e J_e^T)^{-1}] J_e) \delta \theta_{\text{red},j} \\ &= (J_e - J_e J_e^T (J_e J_e^T)^{-1} J_e) \delta \theta_{\text{red},j} = \mathbf{0}. \quad \square \end{aligned}$$

Lemma 2. $\nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T \delta \mathbf{z}_{\text{plc},j} = \mathbf{0}$.

Proof. According to (35) and (36),

$$\begin{aligned} \nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T \delta \mathbf{z}_{\text{plc},j} &= \begin{bmatrix} \nabla_{\theta} \mathbf{e}_{\text{pose}}^T(u_1) G(u_1) \\ \vdots \\ \nabla_{\theta} \mathbf{e}_{\text{pose}}^T(u_l) G(u_l) \end{bmatrix} \begin{bmatrix} G(u_1) \\ \vdots \\ G(u_l) \end{bmatrix}^+ \begin{bmatrix} \delta \theta_{N,j}(u_1) \\ \vdots \\ \delta \theta_{N,j}(u_l) \end{bmatrix} \\ &= \begin{bmatrix} \nabla_{\theta} \mathbf{e}_{\text{pose}}^T(u_1) & & \\ & \ddots & \\ & & \nabla_{\theta} \mathbf{e}_{\text{pose}}^T(u_l) \end{bmatrix} \begin{bmatrix} G(u_1) \\ \vdots \\ G(u_l) \end{bmatrix} \begin{bmatrix} G(u_1) \\ \vdots \\ G(u_l) \end{bmatrix}^+ \begin{bmatrix} \delta \theta_{N,j}(u_1) \\ \vdots \\ \delta \theta_{N,j}(u_l) \end{bmatrix}. \end{aligned} \quad (73)$$

Denoting $\begin{bmatrix} G(u_1) \\ \vdots \\ G(u_l) \end{bmatrix} = \underline{G} \in \mathbb{R}^{l n_{\theta} \times n} (l n_{\theta} > n)$, then the

middle part of (73) is represented by the orthogonal projection matrix $P = \underline{G} \cdot \underline{G}^+$. Considering the structure of $G(u)$ in (13), there exists a permutation matrix H that transforms $\underline{G}$ into the block-diagonal matrix $\hat{G}$, such that:

$$\underline{G} = H \hat{G} = H \begin{bmatrix} \tilde{G}(u_1) \\ \vdots \\ \tilde{G}(u_l) & I \\ & \vdots \\ & I \end{bmatrix}.$$

The pseudo inverse of the block-diagonal $\hat{G}$ can be calculated separately for the blocks, one for the n_j joints of robots and the other for the $n_{\text{pr}} + n_{\text{pw}}$ placement variables. The upper-left block of $\hat{G}$ that consists of l concatenation of matrix $\tilde{G}(u) \in \mathbb{R}^{n_j \times l n_j}$ is invertible due to the B-spline basis. Therefore, the following applies:

$$\begin{aligned} \underline{G} \cdot \underline{G}^+ &= H \hat{G} \cdot (H \hat{G})^+ = H \hat{G} \hat{G}^+ H^T \\ &= H \begin{bmatrix} \begin{bmatrix} \tilde{G}(u_1) \\ \vdots \\ \tilde{G}(u_l) \end{bmatrix} \begin{bmatrix} \tilde{G}(u_1) \\ \vdots \\ \tilde{G}(u_l) \end{bmatrix}^{-1} \\ & \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix} \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix}^+ \end{bmatrix} H^T \\ &= H \begin{bmatrix} I_{l n_j} & \\ & \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix} \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix}^+ \end{bmatrix} H^T. \end{aligned}$$

Then it follows:

$$\begin{aligned}
 \underline{G} \cdot \underline{G}^+ \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \end{bmatrix} &= H \begin{bmatrix} I_{l n_j} & & \\ & \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix} \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix}^+ & \\ & & \end{bmatrix} H^T \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \delta\theta_{N,p_j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \\ \delta\theta_{N,p_j}(u_l) \end{bmatrix} \\
 &= H \begin{bmatrix} I_{l n_j} & & \\ & \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix} \begin{bmatrix} I \\ \vdots \\ I \end{bmatrix}^+ & \\ & & \end{bmatrix} \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \\ \delta\theta_{N,p_j}(u_1) \\ \vdots \\ \delta\theta_{N,p_j}(u_l) \end{bmatrix} = H \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \\ \frac{1}{l} \sum_{i=1}^l \delta\theta_{N,p_j}(u_i) \\ \vdots \\ \frac{1}{l} \sum_{i=1}^l \delta\theta_{N,p_j}(u_i) \end{bmatrix} \\
 &= \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \frac{1}{l} \sum_{i=1}^l \delta\theta_{N,p_j}(u_i) \\ \vdots \\ \delta\theta_{N,j}(u_l) \\ \frac{1}{l} \sum_{i=1}^l \delta\theta_{N,p_j}(u_i) \end{bmatrix}.
 \end{aligned} \tag{74}$$

Since $\delta\theta_{N,j}(u) \in \mathbb{R}^{n_\theta}$ is a basis for the adjustment of the j -th placement variable, $\delta\theta_{N,j}(u)$ changes no other placement variables. Moreover, the entry of the j -th placement in $\delta\theta_{N,j}(u) \in \mathbb{R}^{n_\theta}$ is one, so $\delta\theta_{N,p_j}(u_i)$ are equal to each other for all $i = 1, \dots, l$. Consequently, we have:

$$\frac{1}{l} \sum_{i=1}^l \delta\theta_{N,p_j}(u_i) = \frac{1}{l} \cdot l \cdot \delta\theta_{N,p_j}(u_i) = \delta\theta_{N,p_j}(u_i), \forall i = 1, \dots, l.$$

Thus, (74) can be further written as:

$$\underline{G} \cdot \underline{G}^+ \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \end{bmatrix} = \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \delta\theta_{N,p_j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \\ \delta\theta_{N,p_j}(u_l) \end{bmatrix} = \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \end{bmatrix}.$$

Afterward, Lemma 1 can be utilized to finish the proof:

$$\begin{aligned}
 \nabla_{\mathbf{z}} \mathbf{e}_{\text{pose}}^T \delta \mathbf{z}_{\text{plc},j} &= \begin{bmatrix} \nabla_{\theta} \mathbf{e}_{\text{pose}}^T(u_1) & & \\ & \ddots & \\ & & \nabla_{\theta} \mathbf{e}_{\text{pose}}^T(u_l) \end{bmatrix} \\
 &\quad \begin{bmatrix} \delta\theta_{N,j}(u_1) \\ \vdots \\ \delta\theta_{N,j}(u_l) \end{bmatrix} \\
 &= \mathbf{0}. \quad \square
 \end{aligned}$$

Theorem 4. (The So-Called Mean Value Theorem)

For all $\mathbf{z} \in \mathbb{R}^{n_z}$, $\bar{\mathbf{B}} \in \mathbb{R}^{(n_z - n_e) \times (n_z - n_e)}$, there is:

$$\|\zeta(\mathbf{z}, \bar{\mathbf{B}}) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}) - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}})(\mathbf{z} - \hat{\mathbf{z}})\| \leq \frac{\mu}{2} \|\mathbf{z} - \hat{\mathbf{z}}\|^2.$$

Proof. We utilize Taylor's theorem for vector-valued functions with the Lagrange mean value form of the remainder (Kline, 1998). Taylor's theorem is applied for $\zeta(\mathbf{z}, \bar{\mathbf{B}})$ at the point of expansion $\hat{\mathbf{z}}$:

$$\begin{aligned} & \zeta(\mathbf{z}, \bar{\mathbf{B}}) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}) - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}})(\mathbf{z} - \hat{\mathbf{z}}) \\ &= \frac{1}{2}(\mathbf{z} - \hat{\mathbf{z}})^T \zeta''(\boldsymbol{\eta}_L, \bar{\mathbf{B}})(\mathbf{z} - \hat{\mathbf{z}}), \end{aligned} \quad (75)$$

whereby $\zeta''(\boldsymbol{\eta}_L, \bar{\mathbf{B}})$ is the three-dimensional Hessian tensor. The value of $\boldsymbol{\eta}_L$ is between $\mathbf{z}$ and $\hat{\mathbf{z}}$. Due to (62), we denote $\zeta''(\boldsymbol{\eta}_L, \bar{\mathbf{B}}) = (\nabla \mathbf{d}(\boldsymbol{\eta}_L, \bar{\mathbf{B}}))'$. Inequality (68) is proved by taking norms of (75) and combining with (63). $\square$

Lemma 5. (Relation between $\mathbf{z}_{k-1}$, $\mathbf{z}_k$, $\mathbf{z}_{k+1}$, and $\hat{\mathbf{z}}$)

If the step length of the line search takes $\alpha = 1$ after sufficient number of iterations, there exists a constant $\beta > 0$ such that:

$$\|(\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))(\mathbf{z}_{k+1} - \mathbf{z}_k)\| \leq \beta \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2.$$

Proof. We formulate the left side of (69) to reach a form comparable to the left side of (68) in Theorem 4. As a preparation, we calculate:

$$(\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)\nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_{k-1}).$$

This is done by expanding the expression with products between $\nabla \mathbf{v}(\hat{\mathbf{z}})$, $\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}})$, $\nabla \mathbf{h}_{qp}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)$ and $\nabla \mathbf{h}_{qp}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_{k-1})$ using (54)–(58) and (62). These products have the following properties:

$$\nabla \mathbf{v}(\hat{\mathbf{z}})\nabla \mathbf{v}(\hat{\mathbf{z}}) = -\nabla \mathbf{v}(\hat{\mathbf{z}}), \quad (76)$$

$$\nabla \mathbf{v}(\hat{\mathbf{z}})Z_z(\hat{\mathbf{z}}) = O, \quad (77)$$

$$\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}})Z_z(\hat{\mathbf{z}}) = Z_{qp}Z_{qp}^+ - I, \quad (78)$$

$$Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}})Z_z(\hat{\mathbf{z}})\nabla \mathbf{h}_{qp}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) = O, \quad \text{for } \kappa = k-1, k \quad (79)$$

whereby (76) is a direct result of (55); (77) is a direct result of (55) and (28), whereas (28) is guaranteed by Lemma 2 and (43); (78) is a result of (57), (53), and (77); (79) is a result of (78) and (58). After some lengthy calculation using (76)–(79), it is verified:

$$\begin{aligned} & (\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)\nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_{k-1}) \\ &= \alpha(1 - \alpha)\nabla \mathbf{v}(\hat{\mathbf{z}}). \end{aligned} \quad (80)$$

If the step length of the line search can take $\alpha = 1$ after sufficient iterations, (80) is equal to the zero matrix.

Now we reformulate the left side of (69):

$$\begin{aligned} & (\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))(\mathbf{z}_{k+1} - \mathbf{z}_k) \\ &= (\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))[\zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_k - \hat{\mathbf{z}})] \\ &= (\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))[\zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) \\ &\quad - (I + \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) - \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k))(\mathbf{z}_k - \hat{\mathbf{z}})] \\ &= (\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))[\zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) \\ &\quad - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_k - \hat{\mathbf{z}}) + \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\zeta(\mathbf{z}_{k-1}, \bar{\mathbf{B}}_{k-1}) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_{k-1}))]. \end{aligned}$$

The last equality is valid because of (62). We multiply (80) by $\mathbf{z}_{k-1} - \hat{\mathbf{z}}$ in case of $\alpha = 1$ and add it to the last part:

$$\begin{aligned} & (\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))(\mathbf{z}_{k+1} - \mathbf{z}_k) \\ &= (\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))[\zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) \\ &\quad - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_k - \hat{\mathbf{z}}) \\ &\quad + \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\zeta(\mathbf{z}_{k-1}, \bar{\mathbf{B}}_{k-1}) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_{k-1}) \\ &\quad - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_{k-1})(\mathbf{z}_{k-1} - \hat{\mathbf{z}}))]. \end{aligned}$$

Finally, norms are taken and Theorem 4 is applied twice:

$$\begin{aligned} & \|(\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))(\mathbf{z}_{k+1} - \mathbf{z}_k)\| \\ &\leq \|\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}})\| \\ &\quad \cdot \left[\frac{\mu}{2} \|\mathbf{z}_k - \hat{\mathbf{z}}\|^2 + \|\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)\| \frac{\mu}{2} \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2 \right]. \end{aligned}$$

With the assumption (65) and (66), we further derive:

$$\begin{aligned} & \|(\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}}))(\mathbf{z}_{k+1} - \mathbf{z}_k)\| \\ &\leq \|\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}})\| \left[\frac{\mu}{2} \mu^2 \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2 + \mu \frac{\mu}{2} \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2 \right] \\ &= \frac{\mu^2(1 + \mu)}{2} \|\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}})\| \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2. \end{aligned}$$

We choose $\beta = \mu^2(1 + \mu)/2 \|\nabla \mathbf{v}(\hat{\mathbf{z}}) + Z_z(\hat{\mathbf{z}})\nabla \mathbf{v}_{qp}(\hat{\mathbf{z}})\|$. $\square$

Theorem 6. (Two-Step Linear Convergence of the Reduced Hessian Method)

After sufficient number of iterations, if the active set $\mathcal{I}_a$ of the QP-subproblem is found and no longer changes from iteration to iteration, and if the step length takes $\alpha = 1$, then for each given constant $0 < \rho < 1$, there exist a radius $\varepsilon_1 > 0$ and a reduced Hessian approximation accuracy $\zeta > 0$ such that for $\|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\| \leq \varepsilon_1$, and for

$$\|Z_z(\hat{\mathbf{z}})^T \hat{\mathbf{B}} Z_z(\hat{\mathbf{z}}) - \bar{\mathbf{B}}_k\| < \zeta,$$

the following applies:

$$\|\mathbf{z}_{k+1} - \hat{\mathbf{z}}\| \leq \rho \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|.$$

Proof. The difference between both sides of (71) lies only in the two steps of iteration, so we aim at representing the left side of (71) using $(\mathbf{z}_{k-1} - \hat{\mathbf{z}})$ and some descending steps. We have the following two different expressions for the left side of (71):

$$\mathbf{z}_{k+1} - \hat{\mathbf{z}} = \zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k), \quad (81)$$

$$\mathbf{z}_{k+1} - \hat{\mathbf{z}} = (\mathbf{z}_k - \hat{\mathbf{z}}) + (\mathbf{z}_{k+1} - \mathbf{z}_k). \quad (82)$$

The following way of combining (81) and (82) constitutes the left side of (68):

$$\begin{aligned} & (I - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k))(\mathbf{z}_{k+1} - \hat{\mathbf{z}}) \\ &= \zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_k - \hat{\mathbf{z}}) \\ & \quad - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_{k+1} - \mathbf{z}_k). \end{aligned}$$

Combined with (62), one obtains:

$$\begin{aligned} & -\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_{k+1} - \hat{\mathbf{z}}) \\ &= \zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_k - \hat{\mathbf{z}}) \\ & \quad - (I + \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k))(\mathbf{z}_{k+1} - \mathbf{z}_k) \end{aligned}$$

Lemma 3 provides the inverse:

$$\begin{aligned} & \mathbf{z}_{k+1} - \hat{\mathbf{z}} \\ &= -\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)^{-1} [\zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_k - \hat{\mathbf{z}})] \\ & \quad + (\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)^{-1} + I)(\mathbf{z}_{k+1} - \mathbf{z}_k). \end{aligned} \quad (83)$$

The last term can be extended using Lemma 3, (29), and (53) to reach the form of (69) in Lemma 5:

$$\begin{aligned} & \nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)^{-1} + I \\ &= \underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} - \frac{1}{\alpha} \mathbf{Z}_{\text{qp}}^T \bar{\mathbf{B}}_k \mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+ \right) \\ &= \underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} (\mathbf{Z}_z(\hat{\mathbf{z}}) \mathbf{Z}_z(\hat{\mathbf{z}})^+ - \nabla \mathbf{v}(\hat{\mathbf{z}})) \\ & \quad - \frac{1}{\alpha} \underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \mathbf{Z}_{\text{qp}}^T \bar{\mathbf{B}}_k \mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+ \\ &= \underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \\ & \quad \cdot \left(\mathbf{Z}_z(\hat{\mathbf{z}}) \left[\mathbf{Z}_{\text{qp}} \mathbf{Z}_{\text{qp}}^+ + (\mathbf{A}^T \mathbf{Z}_z(\mathbf{z}_k))^+ \mathbf{A}^T \mathbf{Z}_z(\mathbf{z}_k) \right] \mathbf{Z}_z(\hat{\mathbf{z}})^+ - \nabla \mathbf{v}(\hat{\mathbf{z}}) \right) \\ & \quad - \frac{1}{\alpha} \underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \mathbf{Z}_{\text{qp}}^T \bar{\mathbf{B}}_k \mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+ \\ &= \underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \mathbf{Z}_{\text{qp}}^T \left(\mathbf{Z}_z(\hat{\mathbf{z}})^T \hat{\mathbf{B}} \mathbf{Z}_z(\hat{\mathbf{z}}) - \frac{1}{\alpha} \bar{\mathbf{B}}_k \right) \mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+ \\ & \quad - \underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} (\nabla \mathbf{v}(\hat{\mathbf{z}}) + \mathbf{Z}_z(\hat{\mathbf{z}}) \nabla \mathbf{v}_{\text{qp}}(\hat{\mathbf{z}})). \end{aligned} \quad (84)$$

After combining (84) into (83), we take norms and use the triangle inequality:

$$\begin{aligned} & \|\mathbf{z}_{k+1} - \hat{\mathbf{z}}\| \\ &\leq \|\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)^{-1} [\zeta(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k) - \nabla \zeta(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)(\mathbf{z}_k - \hat{\mathbf{z}})]\| \\ & \quad + \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \mathbf{Z}_{\text{qp}}^T \left(\mathbf{Z}_z(\hat{\mathbf{z}})^T \hat{\mathbf{B}} \mathbf{Z}_z(\hat{\mathbf{z}}) - \frac{1}{\alpha} \bar{\mathbf{B}}_k \right) \mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+(\mathbf{z}_{k+1} - \mathbf{z}_k)\| \\ & \quad + \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} (\nabla \mathbf{v}(\hat{\mathbf{z}}) + \mathbf{Z}_z(\hat{\mathbf{z}}) \nabla \mathbf{v}_{\text{qp}}(\hat{\mathbf{z}}))(\mathbf{z}_{k+1} - \mathbf{z}_k)\|. \end{aligned}$$

The first term is handled with (68) of Theorem 4, the second one with (70), and the third one with (69) from Lemma 5, so we get:

$$\begin{aligned} & \|\mathbf{z}_{k+1} - \hat{\mathbf{z}}\| \\ &\leq \frac{\mu}{2} \|\nabla \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)^{-1}\| \|\mathbf{z}_k - \hat{\mathbf{z}}\|^2 \\ & \quad + \xi \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \mathbf{Z}_{\text{qp}}^T\| \|\mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+\| \|\mathbf{z}_{k+1} - \mathbf{z}_k\| \\ & \quad + \beta \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}}\| \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2. \end{aligned}$$

The first term is further estimated by (67), the second term by (64) with

$$\|\mathbf{z}_{k+1} - \mathbf{z}_k\| = \|\mathbf{d}(\mathbf{z}_k, \bar{\mathbf{B}}_k)\| = \|\mathbf{d}(\mathbf{z}_k, \bar{\mathbf{B}}_k) - \mathbf{d}(\hat{\mathbf{z}}, \bar{\mathbf{B}}_k)\|:$$

$$\begin{aligned} & \|\mathbf{z}_{k+1} - \hat{\mathbf{z}}\| \\ &\leq \frac{\mu^2}{2} \|\mathbf{z}_k - \hat{\mathbf{z}}\|^2 \\ & \quad + \xi \mu \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \mathbf{Z}_{\text{qp}}^T\| \|\mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+\| \|\mathbf{z}_k - \hat{\mathbf{z}}\| \\ & \quad + \beta \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}}\| \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2. \end{aligned}$$

With (65), we get an expression only dependent on $\|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|$ as follows:

$$\begin{aligned} & \|\mathbf{z}_{k+1} - \hat{\mathbf{z}}\| \\ &\leq \left[\frac{\mu^4}{2} + \beta \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}}\| \right] \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|^2 \\ & \quad + \xi \mu^2 \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1} \mathbf{Z}_{\text{qp}}^T\| \|\mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+\| \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|. \end{aligned} \quad (85)$$

We choose an intermediate constant:

$$c_1 \geq \|\underline{\underline{\mathbf{Z}}} \left(\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}} \underline{\underline{\mathbf{Z}}} \right)^{-1}\| \cdot \max\{\|\mathbf{Z}_{\text{qp}}^T\| \|\mathbf{Z}_{\text{qp}} \underline{\underline{\mathbf{Z}}}^+\|, \|\underline{\underline{\mathbf{Z}}}^T \hat{\mathbf{B}}\|\}.$$

For an arbitrary $0 < \rho < 1$, the reduced Hessian approximation accuracy can be assumed to be $\xi = \rho/2c_1\mu^2$ according to Remark 4, and the radius can be chosen as $\varepsilon_1 = \rho/\mu^4 + 2c_1\beta$ with β from Lemma 5. Substituting these choices of constants into (85), we get:

$$\begin{aligned} & \|\mathbf{z}_{k+1} - \hat{\mathbf{z}}\| \\ &\leq \left[\frac{\mu^4}{2} + \beta c_1 \right] \varepsilon_1 \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\| + \xi \mu^2 c_1 \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\| \\ &= \rho \|\mathbf{z}_{k-1} - \hat{\mathbf{z}}\|. \end{aligned}$$

Therefore, we have the two-step linear convergence for the reduced Hessian method in the robotic case with bounding constraints. $\square$

Corollary 7. (Two-Step Superlinear Convergence) After sufficient number of iterations, if the active set $\mathcal{I}_a$ of the QP-subproblem is found and no longer changes from iteration to iteration, if the step length takes $\alpha = 1$, and if the reduced Hessian approximation has

sufficient accuracy as ζ in (70) approaches zero, then the following applies:

$$\lim_{k \rightarrow \infty} \frac{\|\mathbf{z}_{k+1} - \widehat{\mathbf{z}}\|}{\|\mathbf{z}_{k-1} - \widehat{\mathbf{z}}\|} = 0.$$

Proof. By manipulating (85), we obtain:

$$\frac{\|\mathbf{z}_{k+1} - \widehat{\mathbf{z}}\|}{\|\mathbf{z}_{k-1} - \widehat{\mathbf{z}}\|} \leq \left[\frac{\mu^4}{2} + \beta c_1 \right] \|\mathbf{z}_{k-1} - \widehat{\mathbf{z}}\| + \zeta \mu^2 c_1. \quad (86)$$

Due to the convergence, there is $\lim_{k \rightarrow \infty} \|\mathbf{z}_{k-1} - \widehat{\mathbf{z}}\| = 0$. Also considering Remark 4, ζ approaches zero. Therefore, by taking limits on both sides of (86), we obtain (72). $\square$